

Convolutional Neural Networks

Hrachya Asatryan

Computer Vision

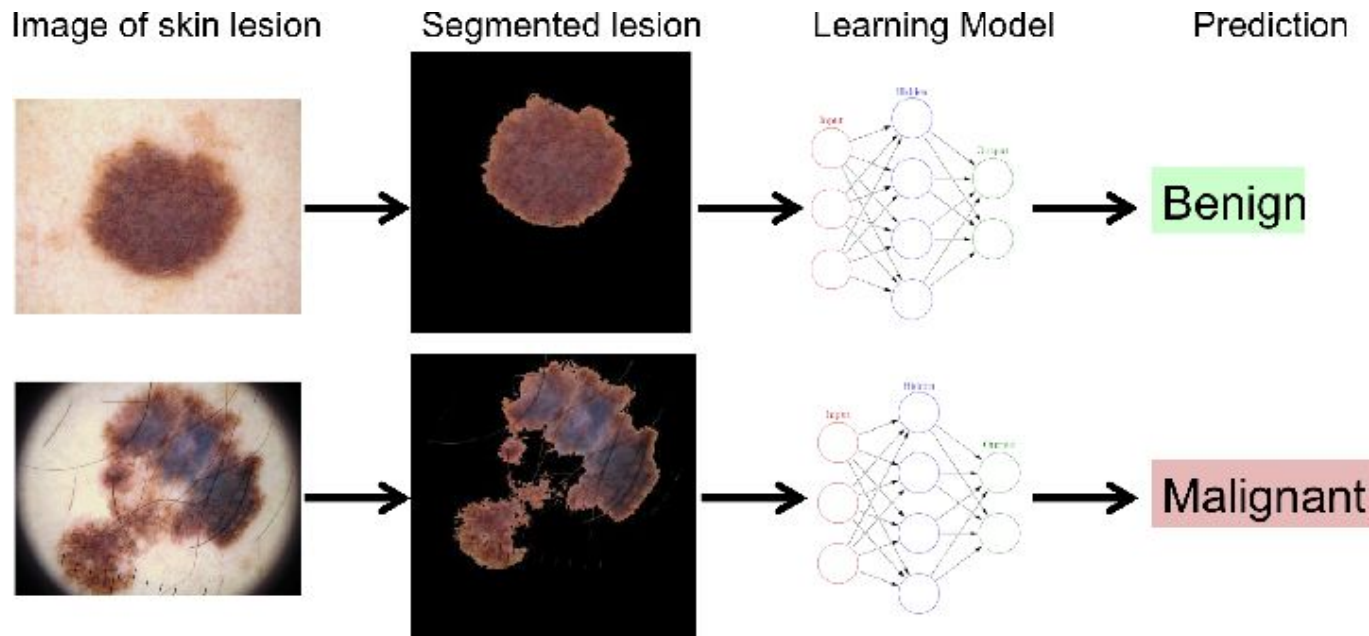
Computer vision (CV) is a field of machine learning (ML) that uses machine learning and neural networks to teach computers and systems to derive meaningful information from digital images, videos and other visual inputs—and to make recommendations or take actions when they see defects or issues.

Computer vision tasks include methods for acquiring, processing, analyzing and understanding digital images, and extraction of high-dimensional data from the real world in order to produce numerical or symbolic information, e.g. in the forms of decisions.

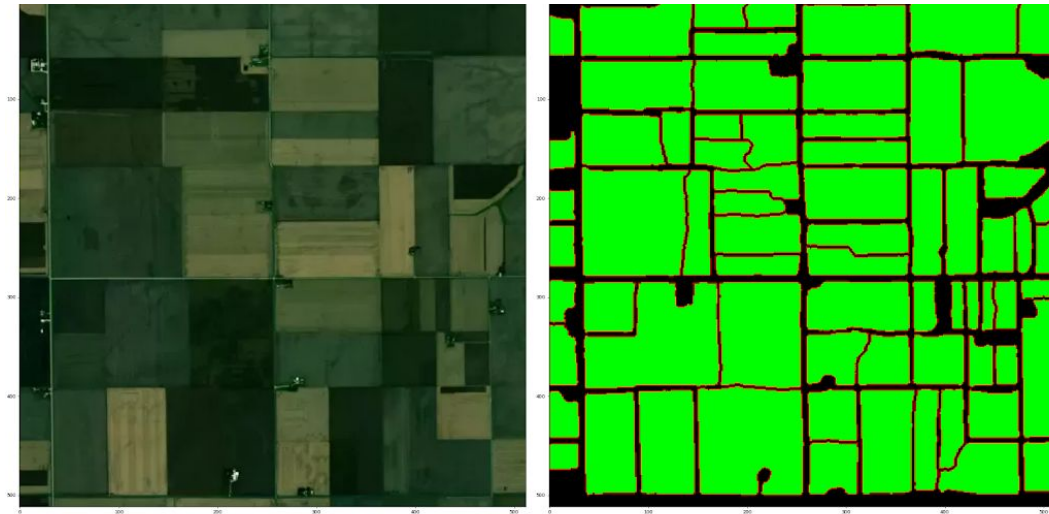
Applications of CV



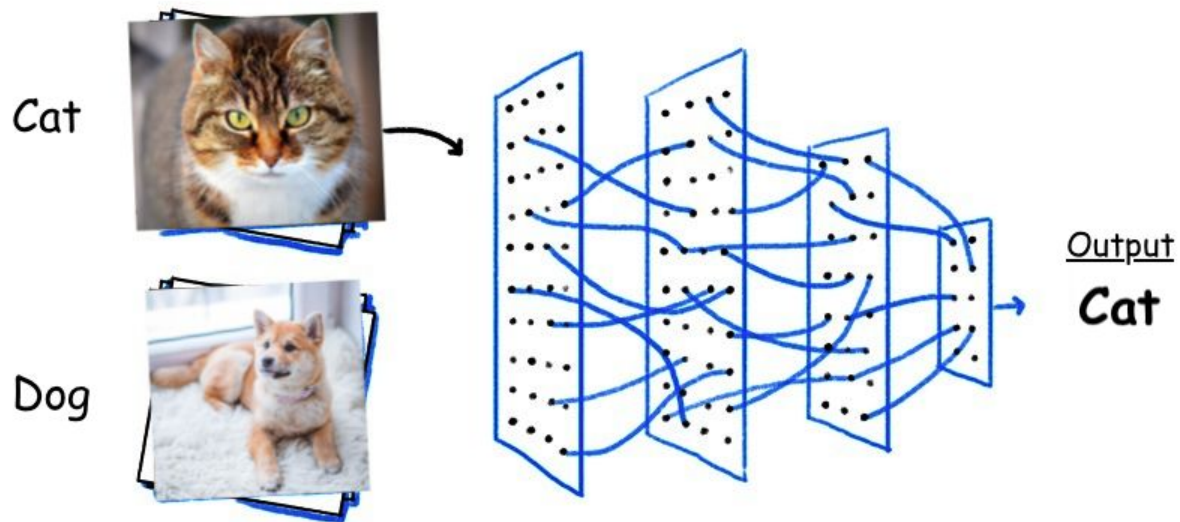
Applications



Applications



Applications



Convolutional Neural Networks

A Convolutional Neural Network (CNN) is a deep learning algorithm designed specifically to analyze visual data. Inspired by the organization of the animal visual cortex, CNNs are adept at recognizing patterns in images through a hierarchical structure of interconnected layers. Its key components are:

- Convolutional Layers: These layers apply filters to input images, extracting features such as edges, textures, and shapes.
- Pooling Layers: Pooling layers downsample the feature maps, reducing computational complexity while retaining important information.
- Fully Connected Layers: These layers perform classification based on the features extracted by the preceding layers.

CNNs are highly effective in tasks like image classification, object detection, and segmentation, making them essential tools in computer vision and beyond.

Visualization

Convolutional Neural Networks

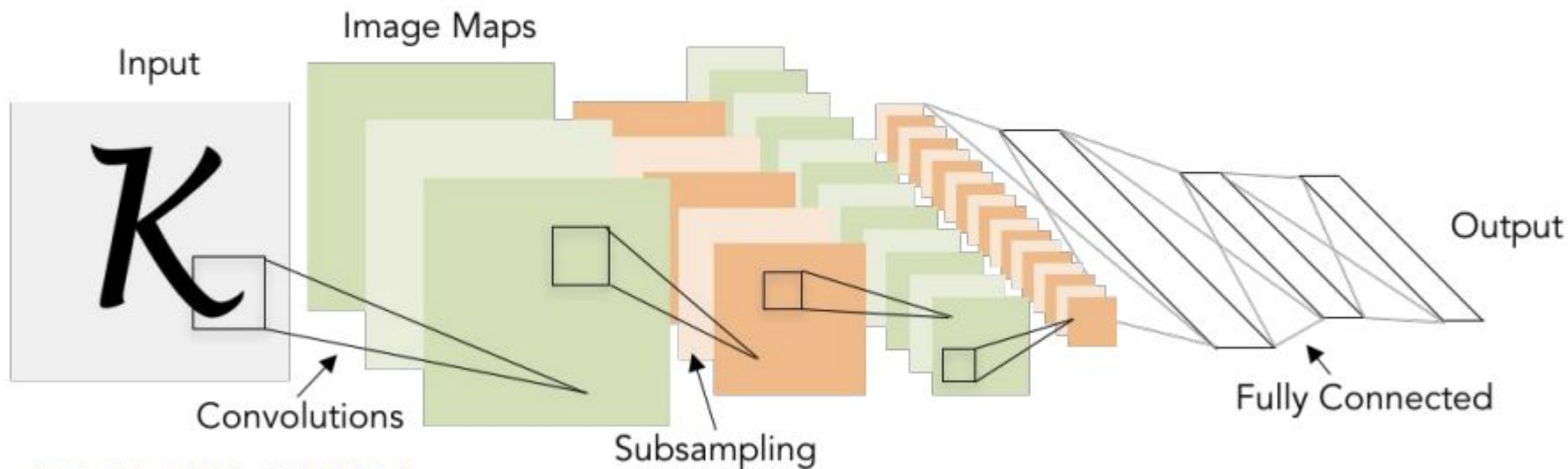


Illustration of LeCun et al. 1998 from CS231n 2017 Lecture 1

Convolutions

Input

0	1	2
3	4	5
6	7	8

Kernel

0	1
2	3

*

=

Output

19	25
37	43

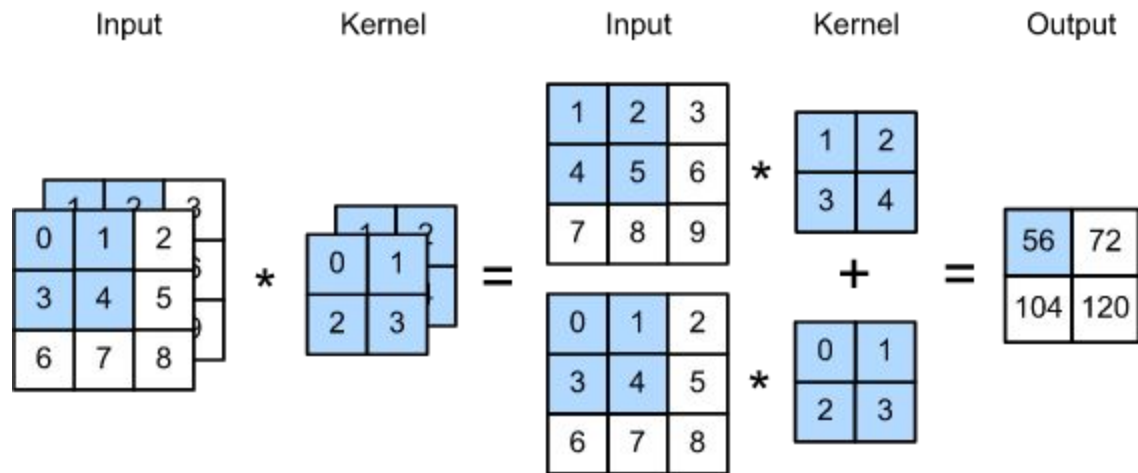
$$0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 = 19,$$

$$1 \times 0 + 2 \times 1 + 4 \times 2 + 5 \times 3 = 25,$$

$$3 \times 0 + 4 \times 1 + 6 \times 2 + 7 \times 3 = 37,$$

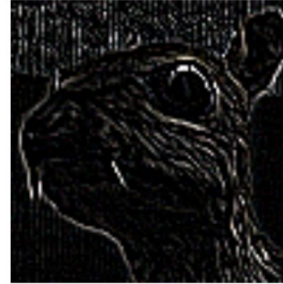
$$4 \times 0 + 5 \times 1 + 7 \times 2 + 8 \times 3 = 43.$$

Convolutions

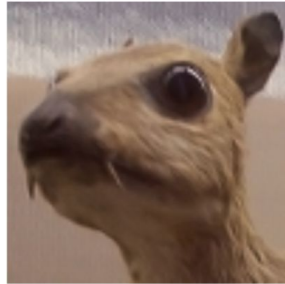


Examples

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

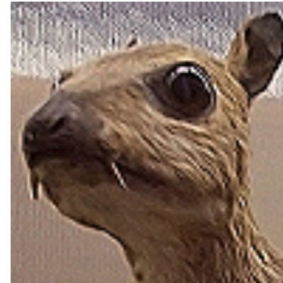


Edge Detection



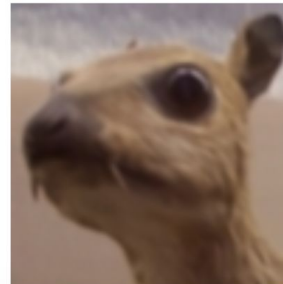
(wikipedia)

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$



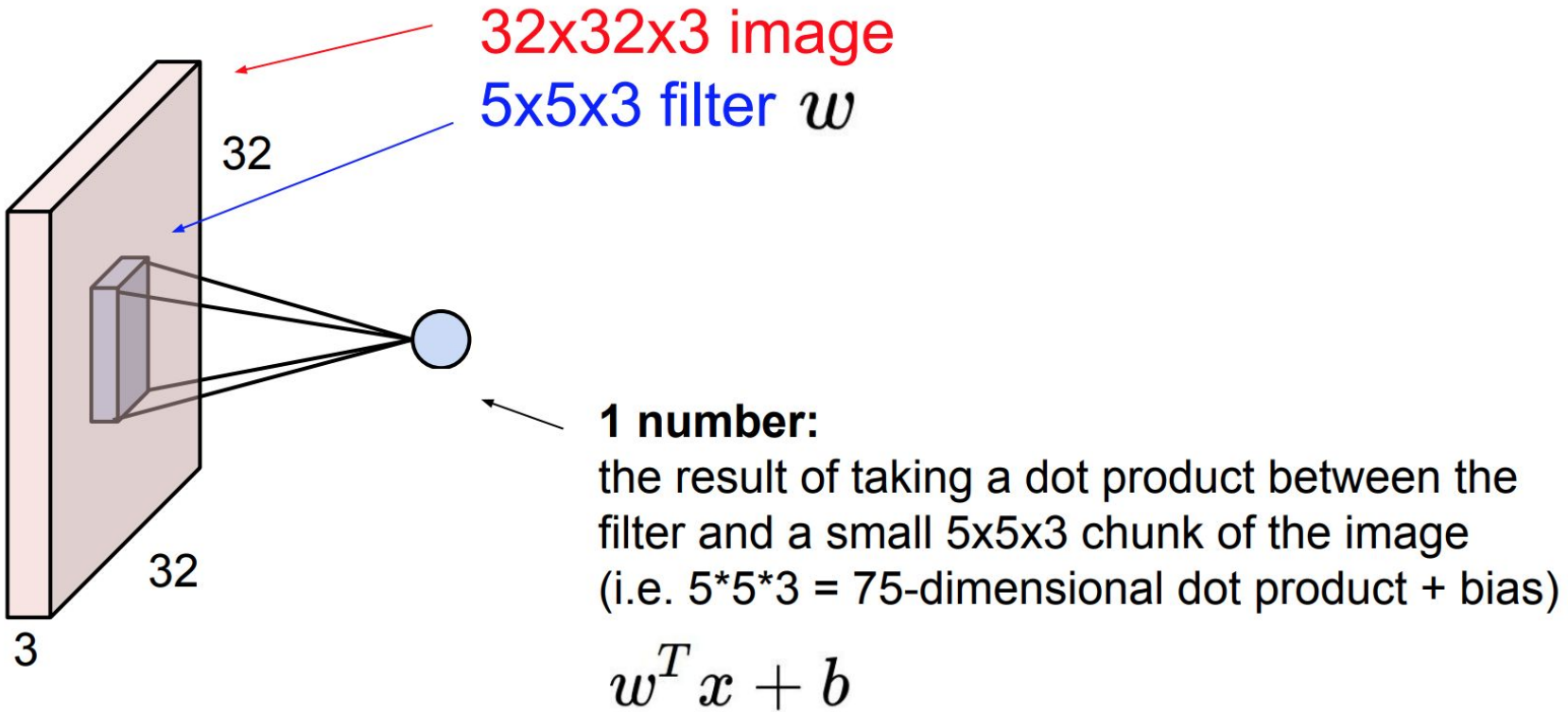
Sharpen

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

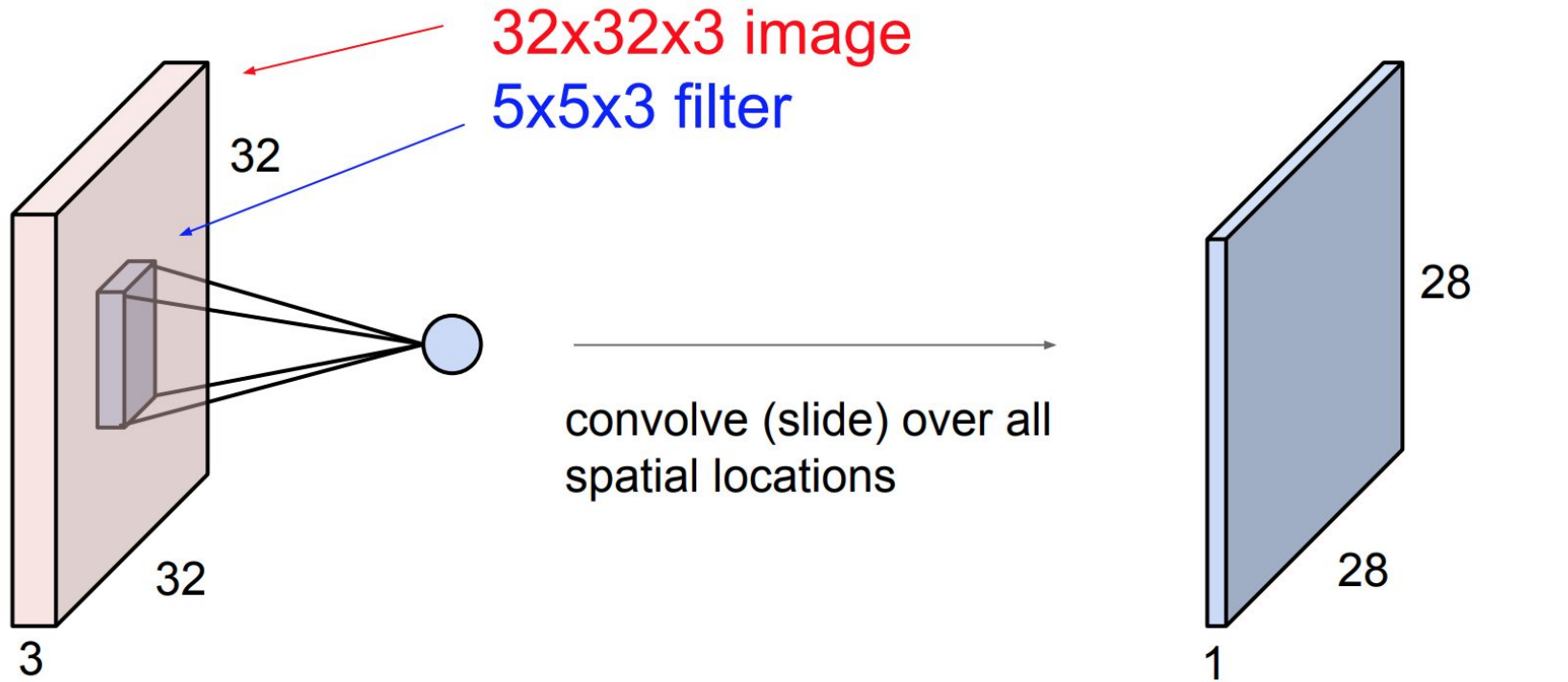


Gaussian Blur

Convolutional Layer

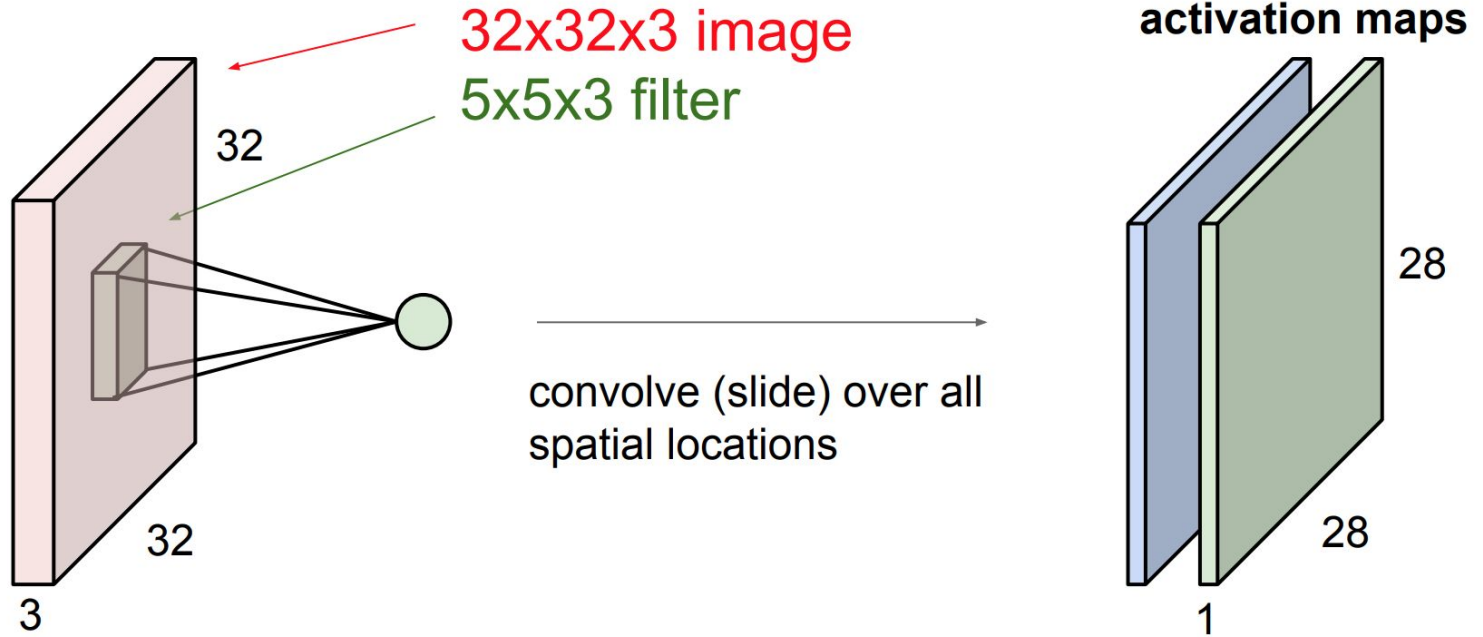


Convolutional Layer



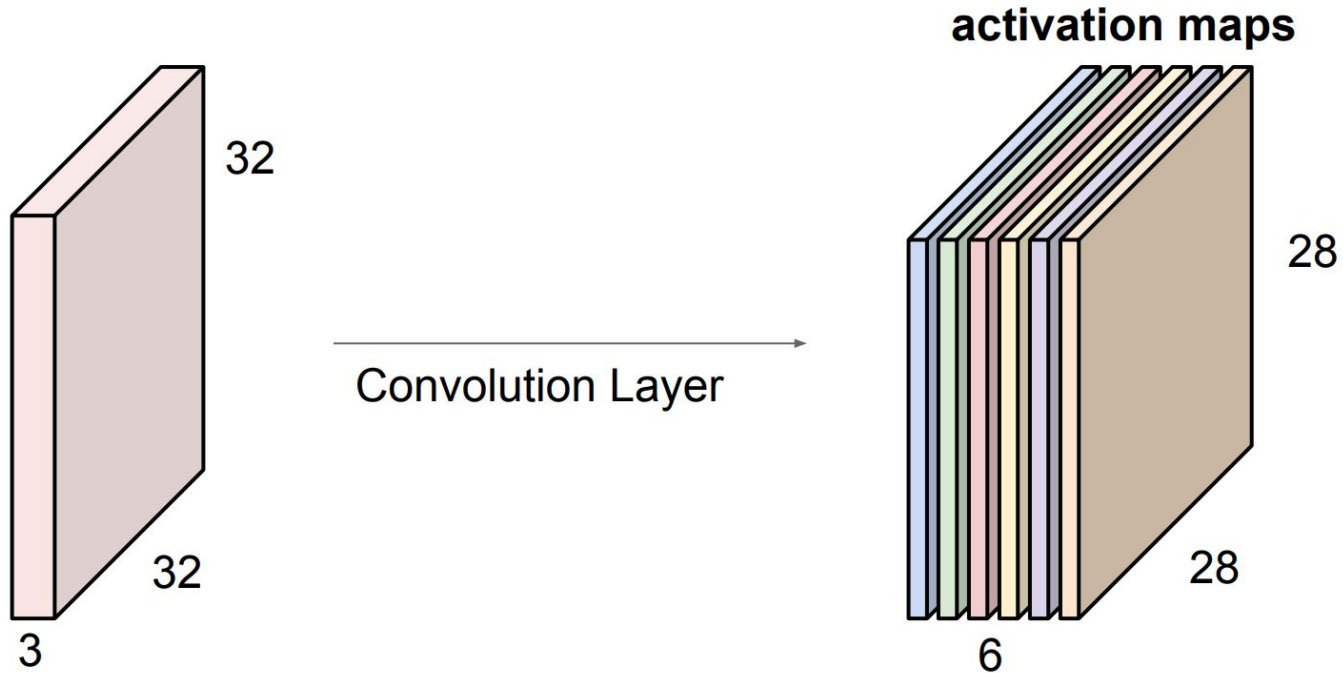
Convolutional Layer

consider a second, **green** filter



Convolutional Layer

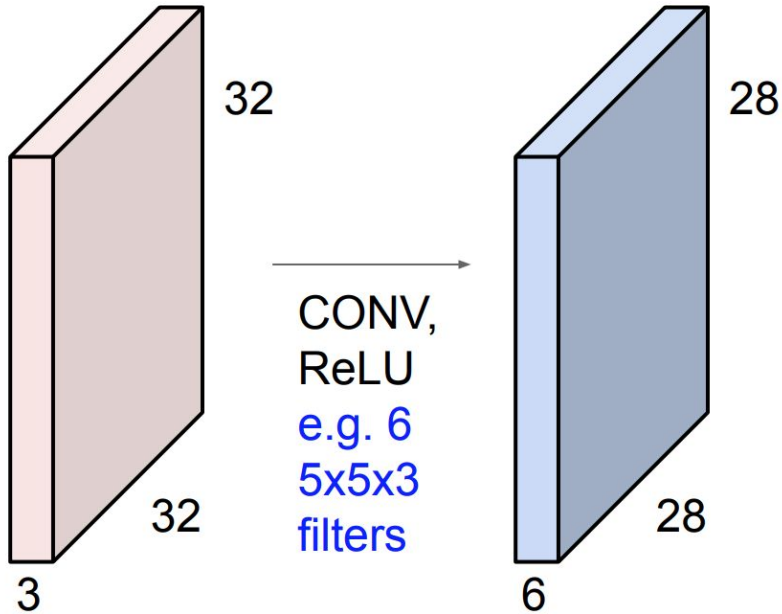
For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



We stack these up to get a “new image” of size 28x28x6!

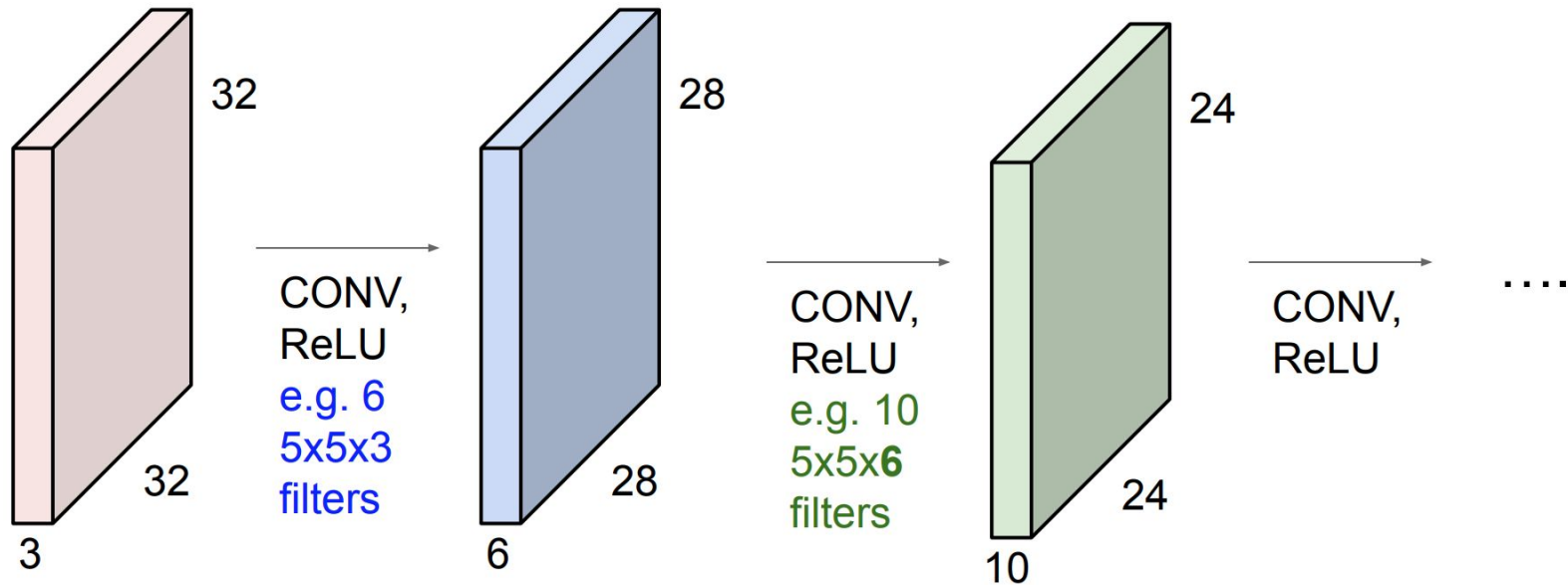
ConvNet

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



ConvNet

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



Padding

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

114				

Pooling

Max Pooling

29	15	28	184
0	100	70	38
12	12	7	2
12	12	45	6

2 x 2
pool size

100	184
12	45

Average Pooling

31	15	28	184
0	100	70	38
12	12	7	2
12	12	45	6

2 x 2
pool size

36	80
12	15

Max Pooling

Take the **highest** value from the area covered by the kernel

Average Pooling

Calculate the **average** value from the area covered by the kernel

Example: Kernel of size 2 x 2; stride=(2,2)

3	2	0	0
0	7	1	3
5	2	3	0
0	9	2	3

Convolved
Feature
(4 x 4)

Output

Max
values

7	

3	2	0	0
0	7	1	3
5	2	3	0
0	9	2	3

Convolved
Feature
(4 x 4)

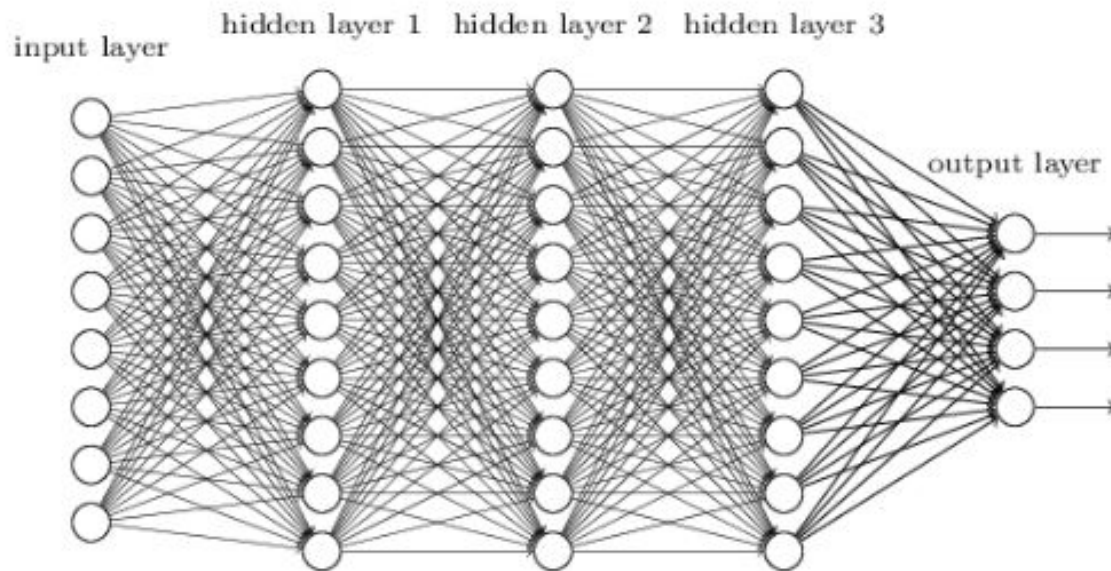
Output

Average
values

3	

Convolutional Layer

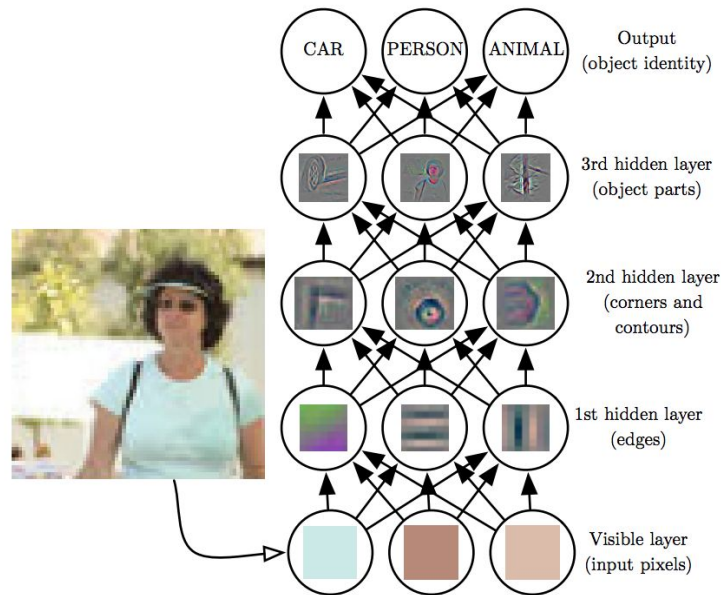
Consider this fully connected model



From this fully connected model, do we really need all the edges? Can some of these be shared?

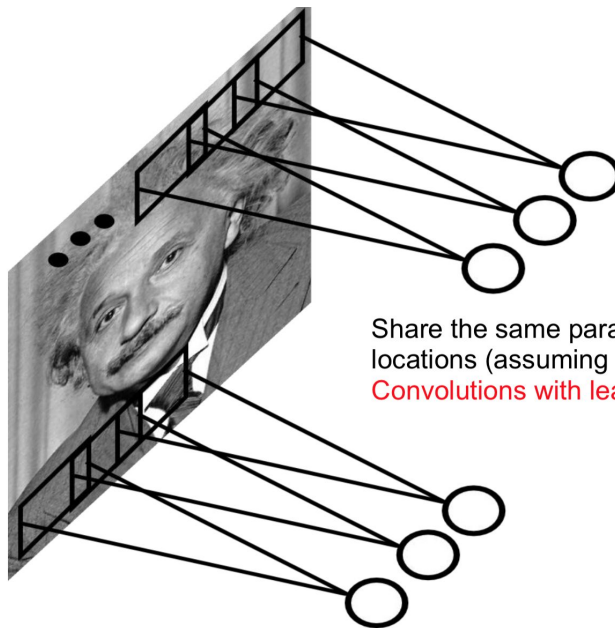
Feature Learning

Through successive layers of convolutional and pooling operations, CNNs can capture increasingly complex features, starting from simple edges and textures to more abstract concepts like shapes and objects.



Weight Sharing

Parameter sharing greatly reduces the number of parameters compared to fully connected networks, making CNNs more efficient to train and less prone to overfitting. Additionally, it helps CNNs generalize better to variations in input data, such as translation, rotation, and scale changes.

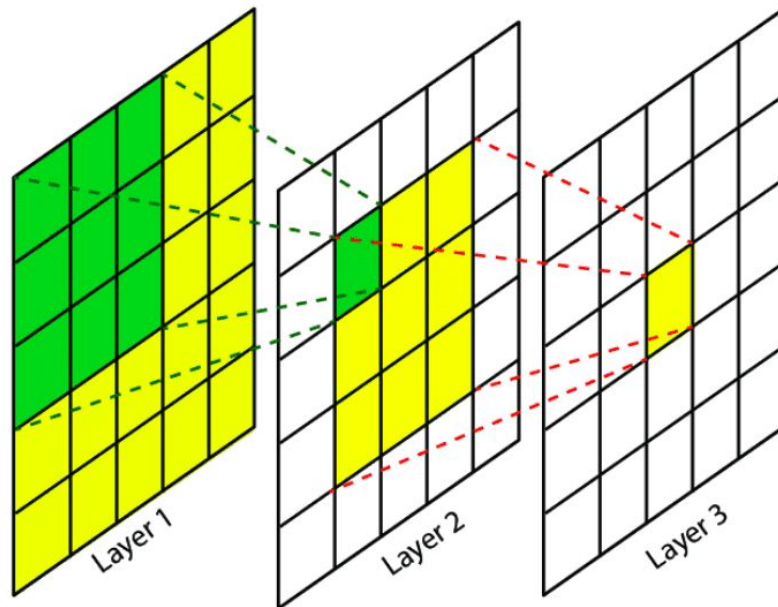


Share the same parameters across different locations (assuming input is stationary):

Convolutions with learned kernels

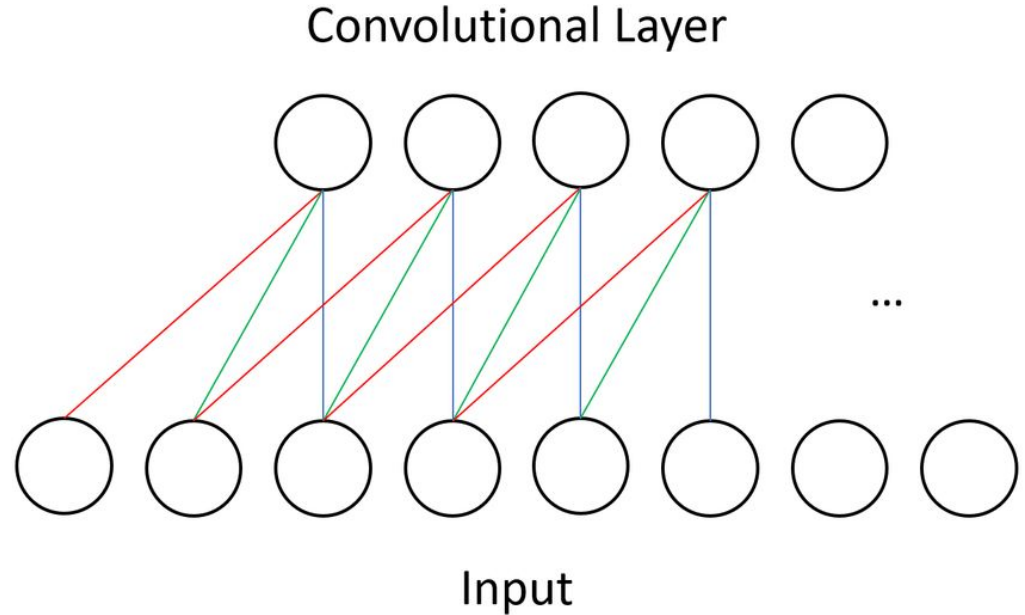
Local Receptive Field

By focusing on small regions of the input at a time and sharing parameters across these regions, CNNs can effectively capture local patterns and spatial relationships in the data. This allows CNNs to achieve translation invariance, meaning they can recognize objects regardless of their position within the input image.

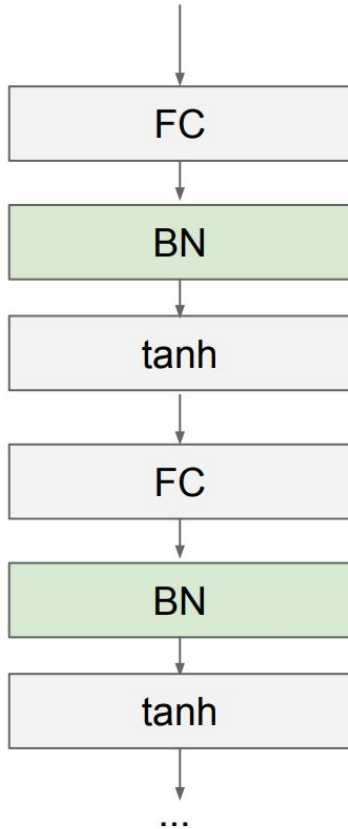


Sparse connectivity

In CNNs, each neuron in a convolutional layer is connected only to a small subset of neurons in the previous layer, determined by the size of the convolutional filters. This sparse connectivity reduces the computational complexity of the network while still allowing it to capture meaningful patterns in the data.



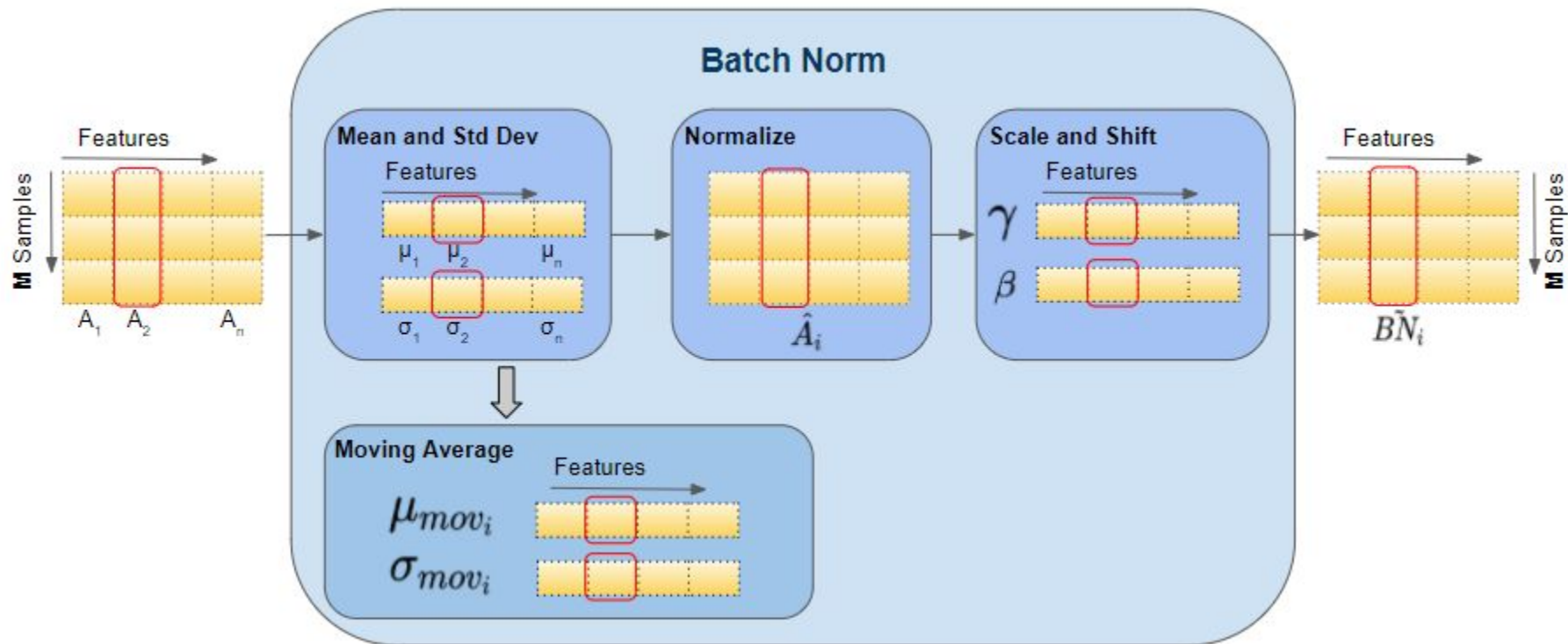
Batch Normalization



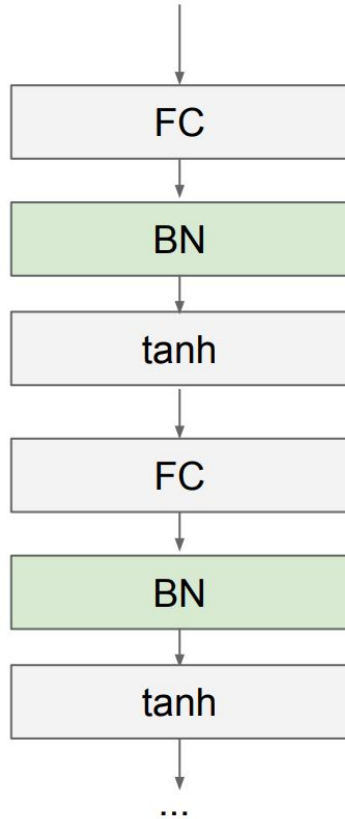
Usually inserted after Fully Connected or Convolutional layers, and before nonlinearity.

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

Batch Norm



Batch Norm



- Makes deep networks **much** easier to train!
- Improves gradient flow
- Allows higher learning rates, faster convergence
- Networks become more robust to initialization
- Acts as regularization during training
- Zero overhead at test-time: can be fused with conv!
- Behaves differently during training and testing: this is a very common source of bugs!

The Problem with Batch Norm

- **Key Concept:** Batch Norm relies on accurate statistics (mean/variance) calculated from the current mini-batch.
- **The Issue:** If your batch size is small (e.g., 1 or 2 images per GPU because the model is huge or image resolution is high, like in Medical Imaging or Object Detection), the statistics are noisy. This leads to high training error.

Alternative Normalization Methods

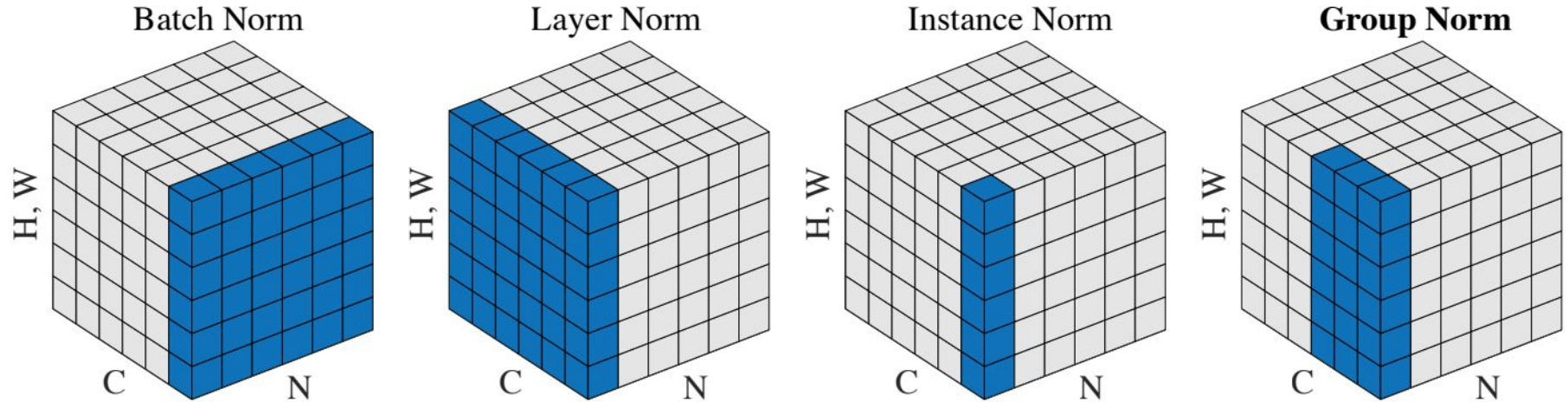
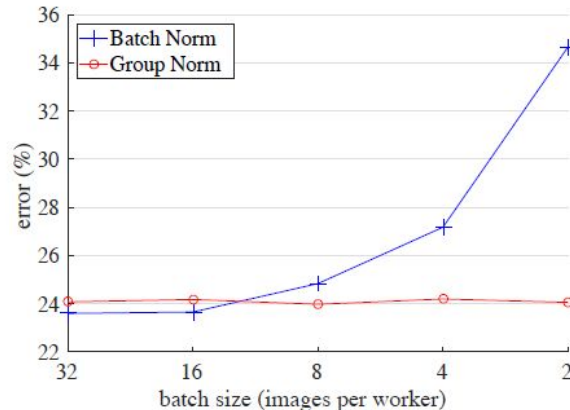


Figure 2. **Normalization methods.** Each subplot shows a feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels.

Group Norm

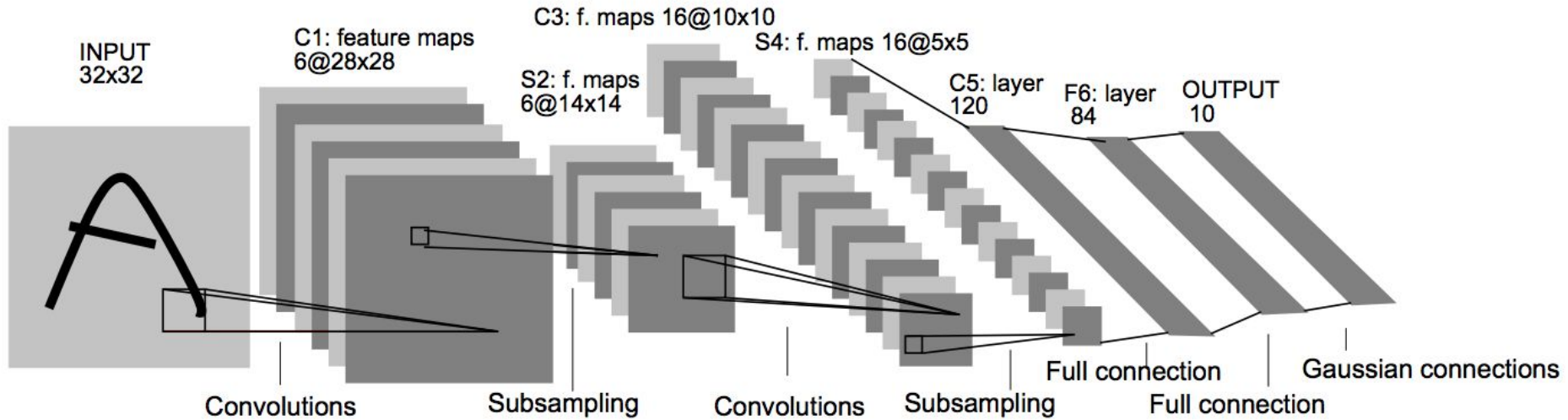
- Key Concept: GN is independent of batch size.
- Mechanism: It separates channels into groups (e.g., 32 channels -> 4 groups of 8). It calculates mean/variance for each group within a single sample.
- Why it works: Channels in CNNs often learn correlated features (e.g., different orientations of edges). Grouping them allows the model to normalize based on "feature types" rather than the whole layer.



History of CNNs

LeNet by Yann Lecun - 1998

Parameters: 60,000

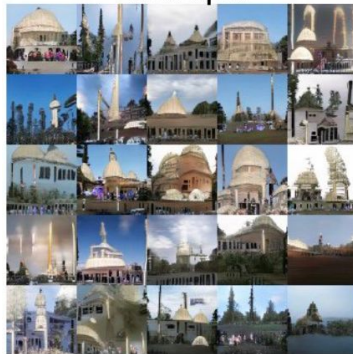


ImageNet dataset

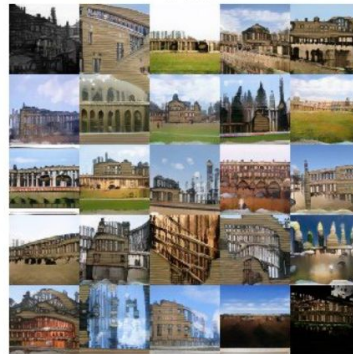


1000 classes and
1M+ images

Mosque



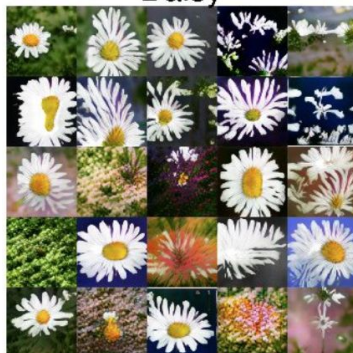
Palace



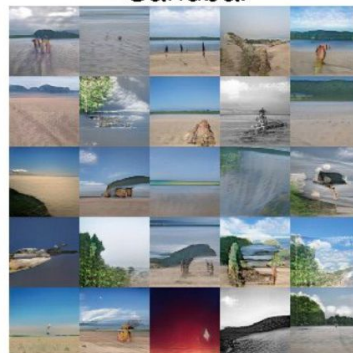
Schooner



Daisy



Sandbar

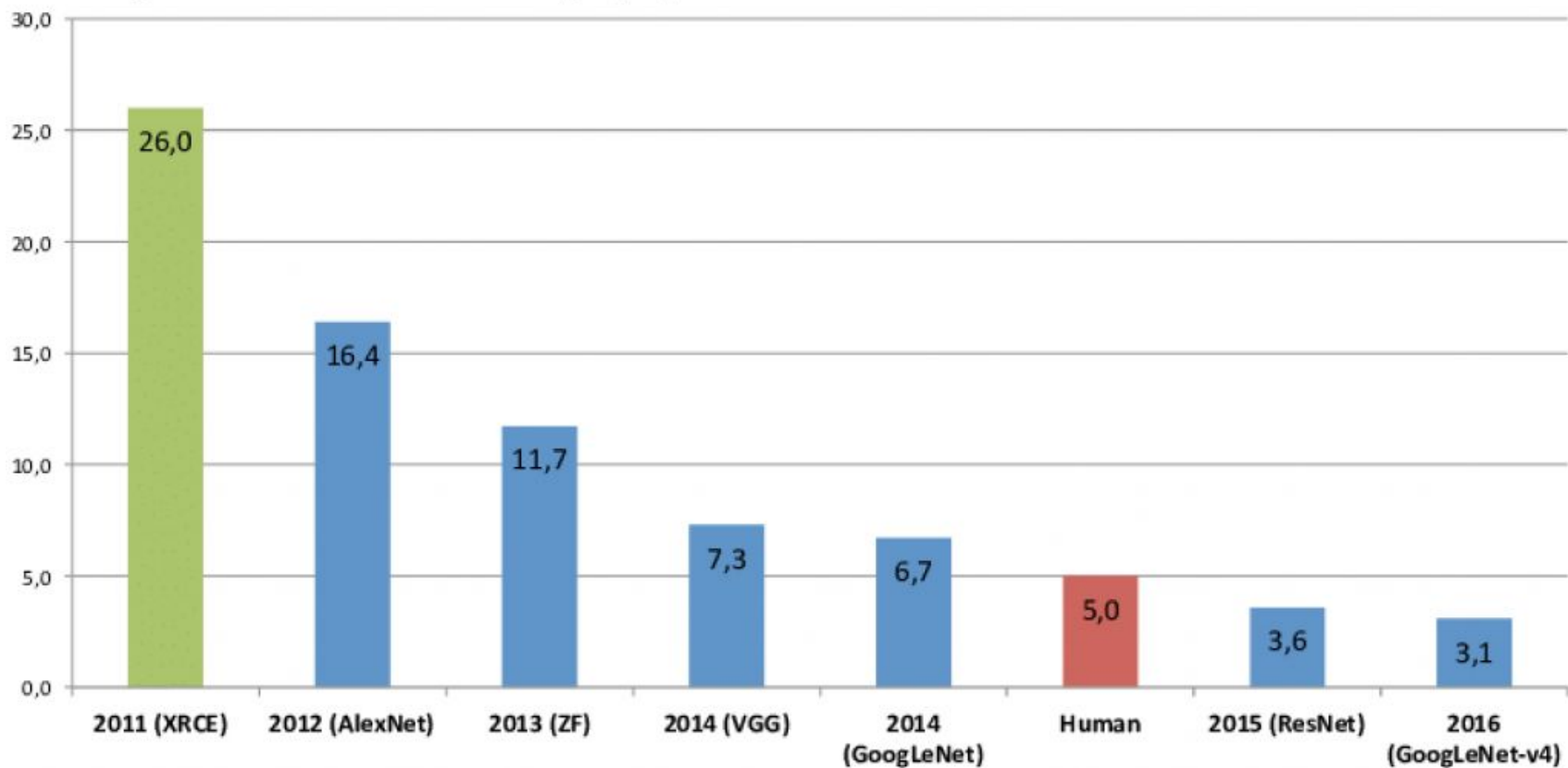


Pizza



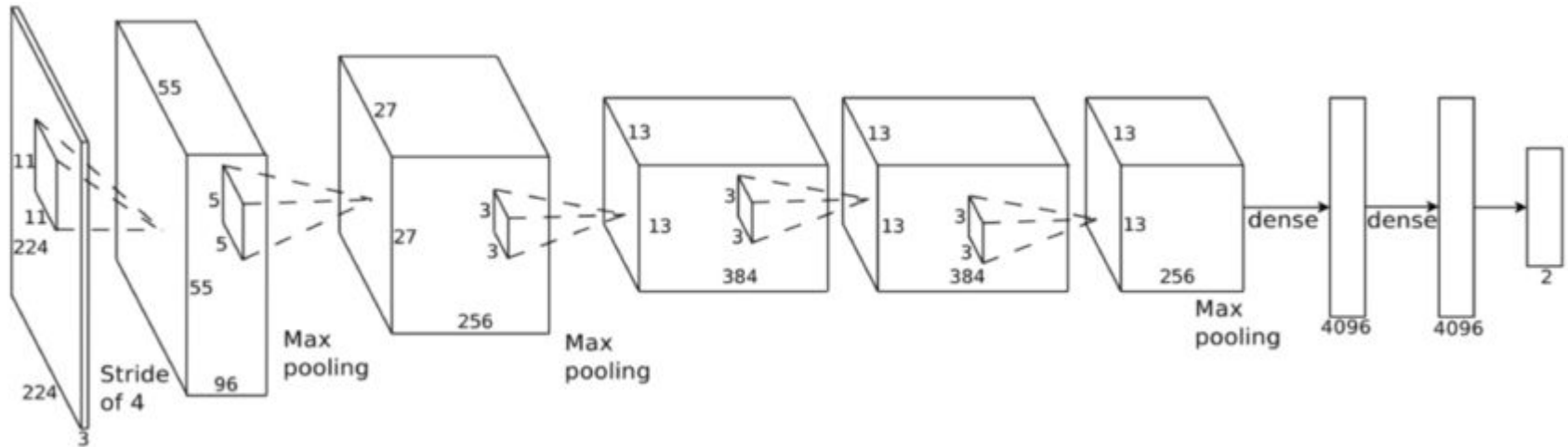
ImageNet Results

ImageNet Classification Error (Top 5)

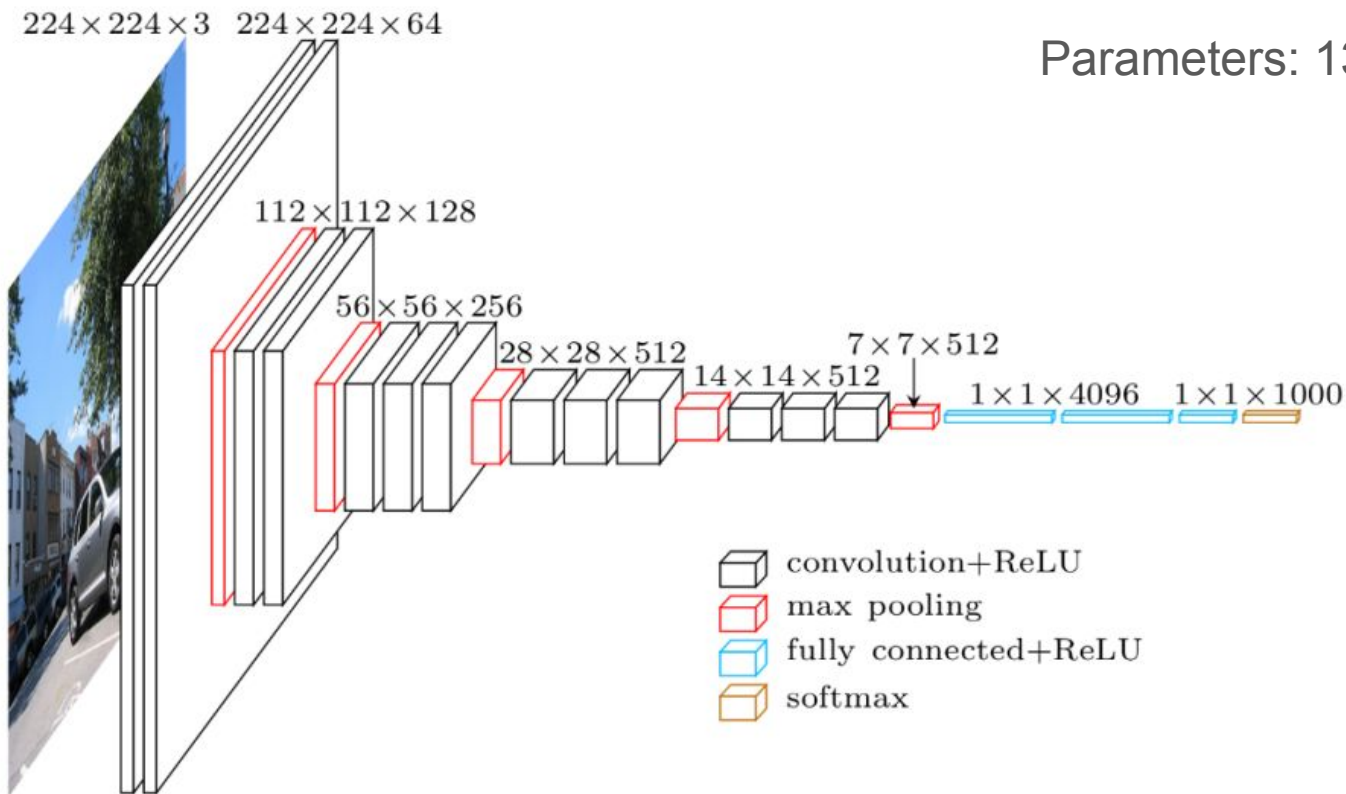


AlexNet by Alex Krizhevsky et al. in 2012

Parameters: 60 million



VGG by Karen Simonyan et al. in 2014



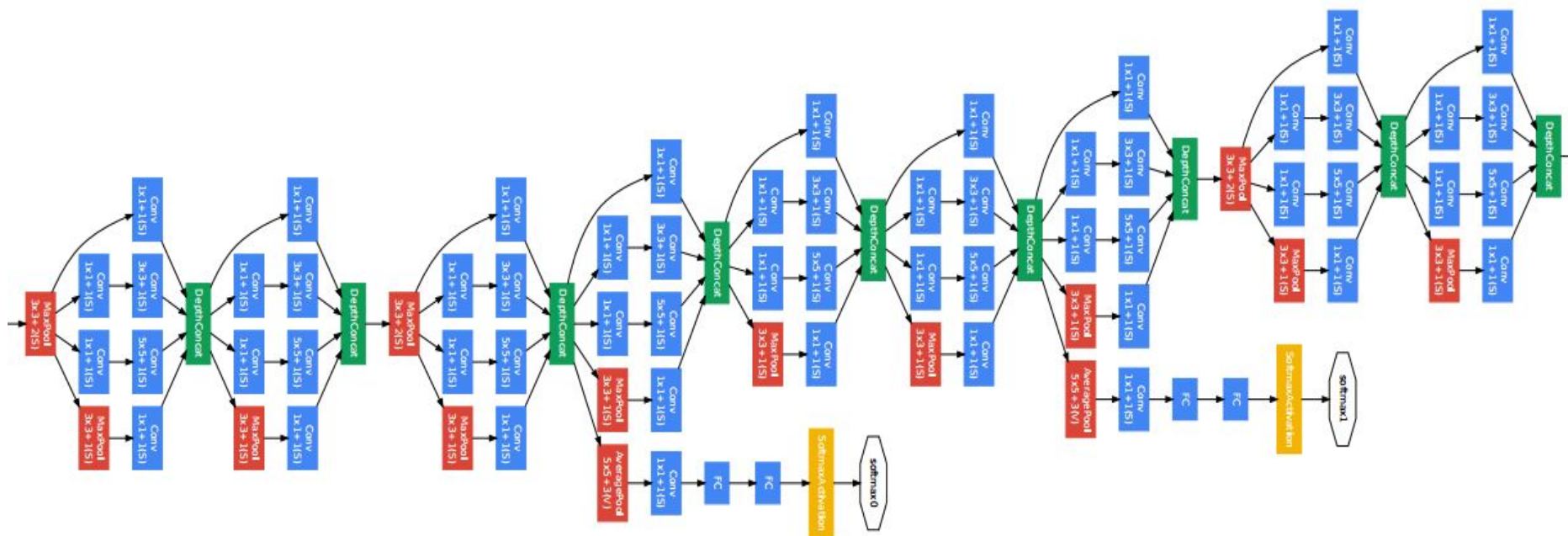
VGG-19

Top-1 Accuracy: 70.5%

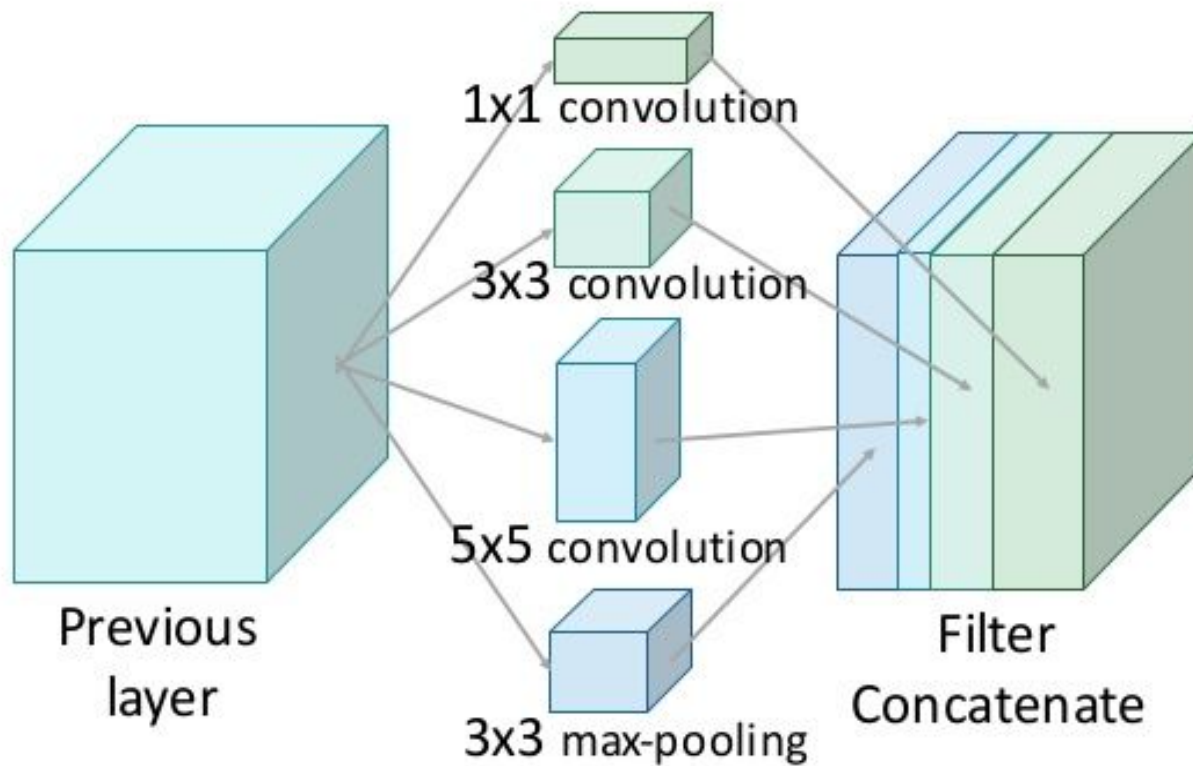
Top-5 Accuracy: 90.0%



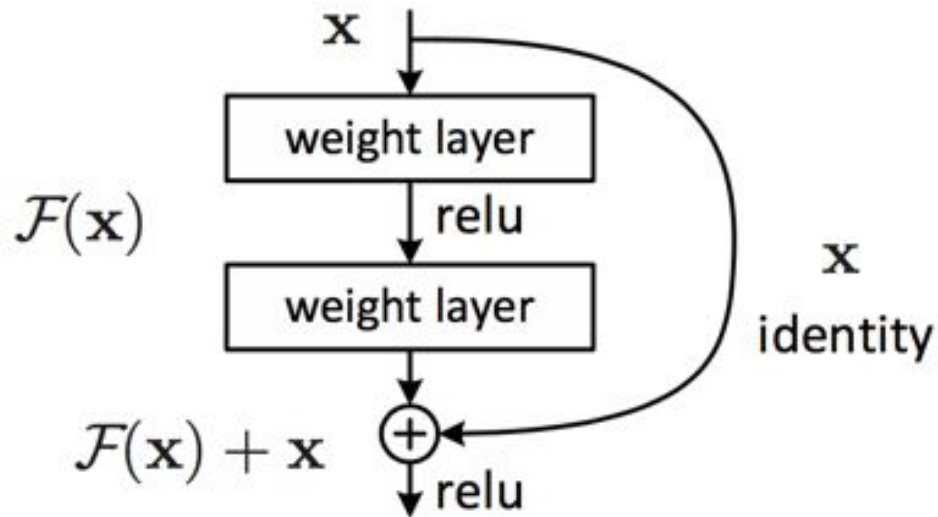
GoogleNet (InceptionNet)



InceptionNet Module

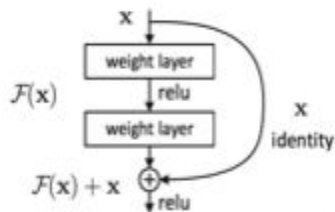


Residual Block (skip connection)

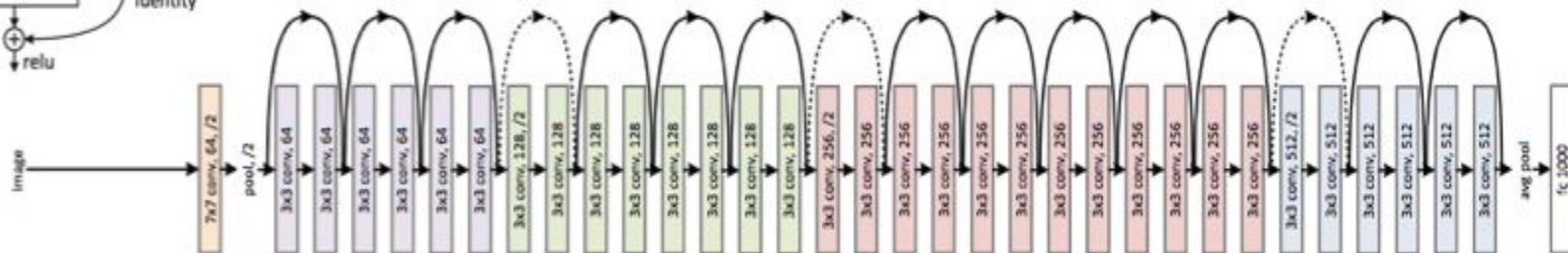


ResNet

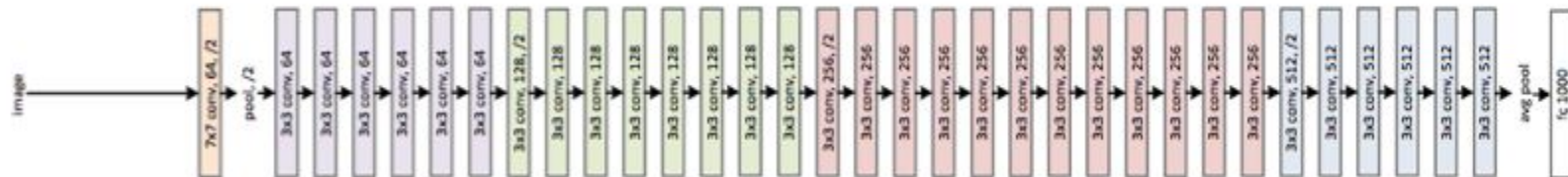
residual connection



Residual Network



Plain Network



L2 Regularization in CNNs

Key Concept: Penalizing large weights in Convolutional Kernels.

CNN Specifics:

- Without L2, kernels can become "noisy" or overly sharp to fit specific pixels.
- With L2: Kernels tend to be smoother and more distinct (Gabor-like filters).

Dropout in CNNs

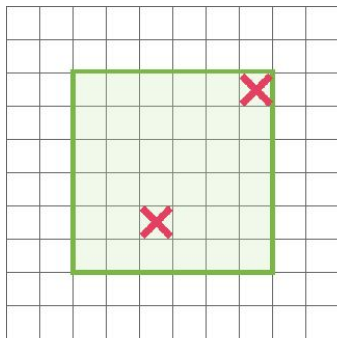
Key Concept: Standard Dropout turns off random neurons (pixels in feature maps).

The Problem with Standard Dropout in Conv Layers:

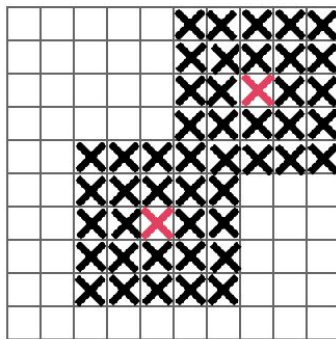
- Pixels in feature maps are spatially correlated. If you drop pixel (i, j) , the network can still easily infer its value from neighbor $(i, j+1)$.
- *Result:* Standard dropout is weak for Conv layers; it mostly just slows down training without much regularization benefit.

Dropout in CNNs

- **Key Concept:** Instead of dropping random pixels, drop entire *channels* or contiguous *blocks* of the feature map.
- **Spatial Dropout:** Drop the entire k-th feature map for a sample. Forces the network to not rely on that specific feature (e.g., "beak detector") at all.
- **DropBlock:** Drop a square region e.g., 5x5 block) of the feature map. Forces the network to look at other parts of the object.



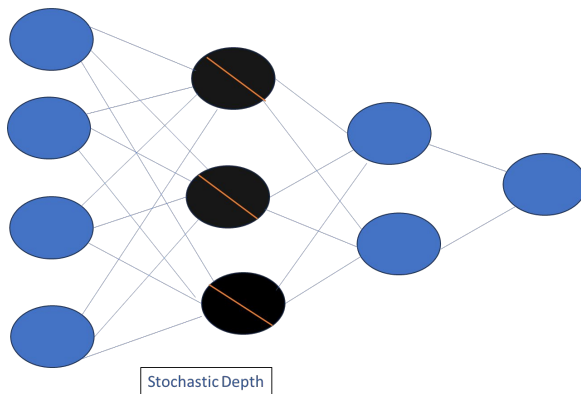
(a)



(b)

Stochastic Depth

- **Key Concept:** Used specifically for ResNets (in general applicable to only deep networks)
- **Mechanism:** Randomly "skip" entire Residual Blocks during training.
 - a. Training: The network is effectively shallower (subset of layers).
 - b. Testing: Use the full deep network.
- **Benefit:** Acts like an ensemble of many shallower networks.



Data Augmentation

The Goal: Invariance

- **Key Concept:** A "dog" is still a "dog" if it is on the left side, right side, or slightly darker.
- **Goal:** We want our CNN to be *invariant* to translation, viewpoint, size, and illumination.



Geometric Augmentation

- **Key Concept:** Physically moving pixels.
- **Techniques:**
 - a. **Random Resized Crop (The Gold Standard):** Pick a random patch of the image, resize it back to 224x224. This forces the network to learn "parts" of the object (e.g., just the ear) and the "whole".
 - b. **Horizontal Flip:** Valid for natural scenes (not always valid for text/numbers!).

Random Resized Crop

Random Resized Crop
(One image → Infinite variations)



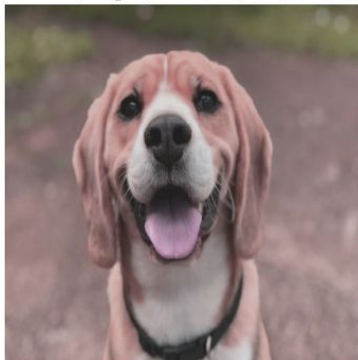
Photometric Augmentations

- **Key Concept:** Changing pixel values, not positions.
- **Techniques:**
 - **Color Jitter:** Randomly change Brightness, Contrast, Saturation, Hue.
 - **Grayscale:** Randomly convert to B&W.
- **Why:** Prevents the network from relying on color alone (e.g., "strawberries are always red").

Original



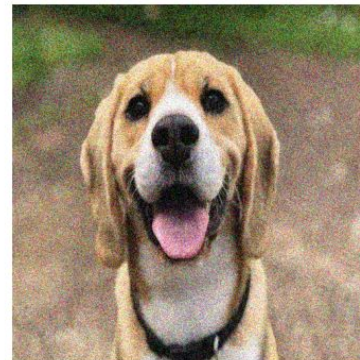
Color jitter
(Bright/Contrast/Sat)



Grayscale



Gaussian Noise



Advanced Augmentation

These are Modern "Strong" Augmentations (State-of-the-Art).

- **CutOut:** Randomly mask out a black square in the image. (Forces network to not look at just the most discriminative part).
- **MixUp:** Take two images (Cat, Dog). Linearly blend them: $0.7 \times \text{Cat} + 0.3 \times \text{Dog}$. Label becomes $[0.7, 0.3]$.
 - **Result:** The decision boundary becomes smoother/linear.
- **CutMix:** Cut a patch from a Cat image and paste it onto a Dog image. Label is proportional to area pixels.

Modern Augmentations

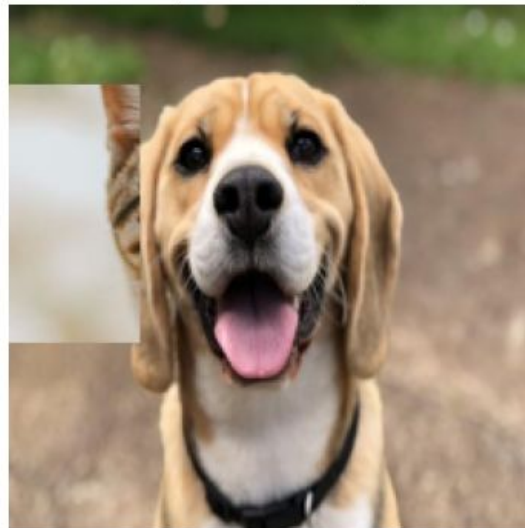
CutOut
(Region Dropout)



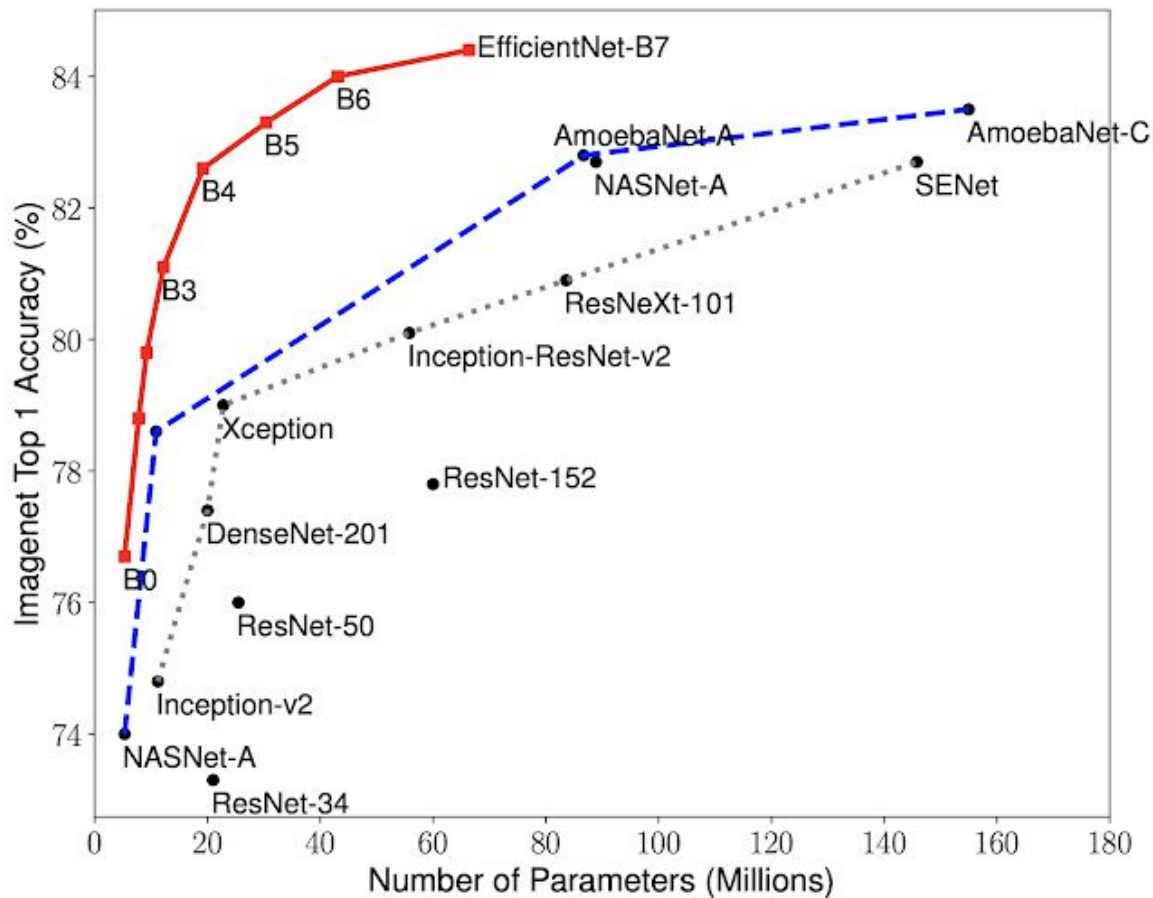
MixUp
($0.6 \cdot \text{Dog} + 0.4 \cdot \text{Cat}$)



CutMix
(Patch Paste)

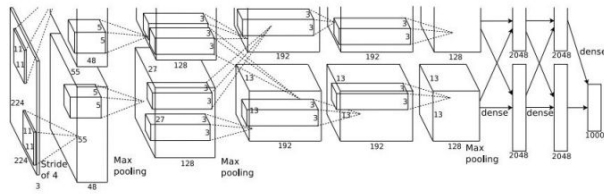
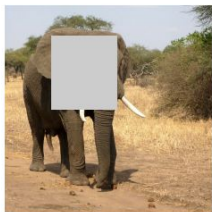
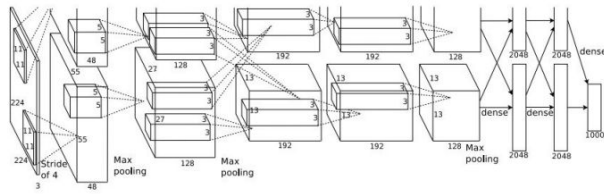


ImageNet Comparison

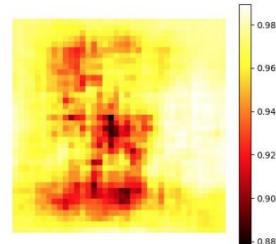


Which Pixels Matter: Saliency vs Occlusion

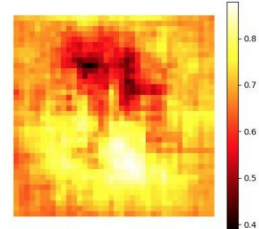
Mask part of the image before feeding to CNN,
check how much predicted probabilities change



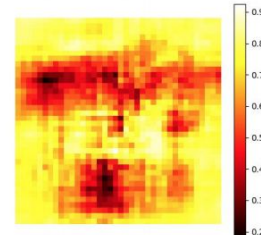
schooner



African elephant, *Loxodonta africana*

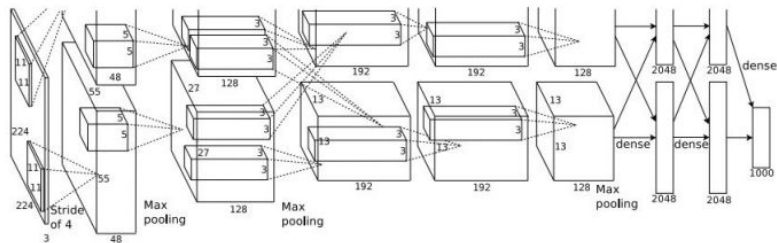


go-kart



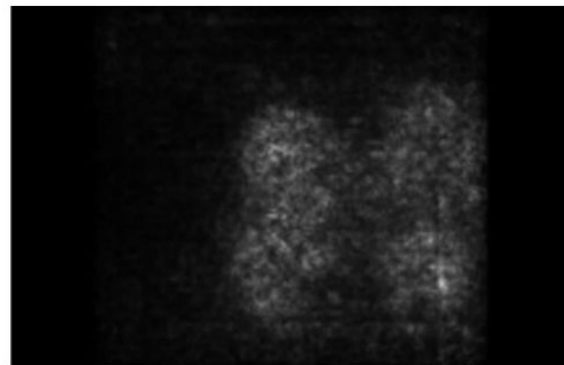
Which Pixels Matter: Saliency vs Backprop

Forward pass: Compute probabilities

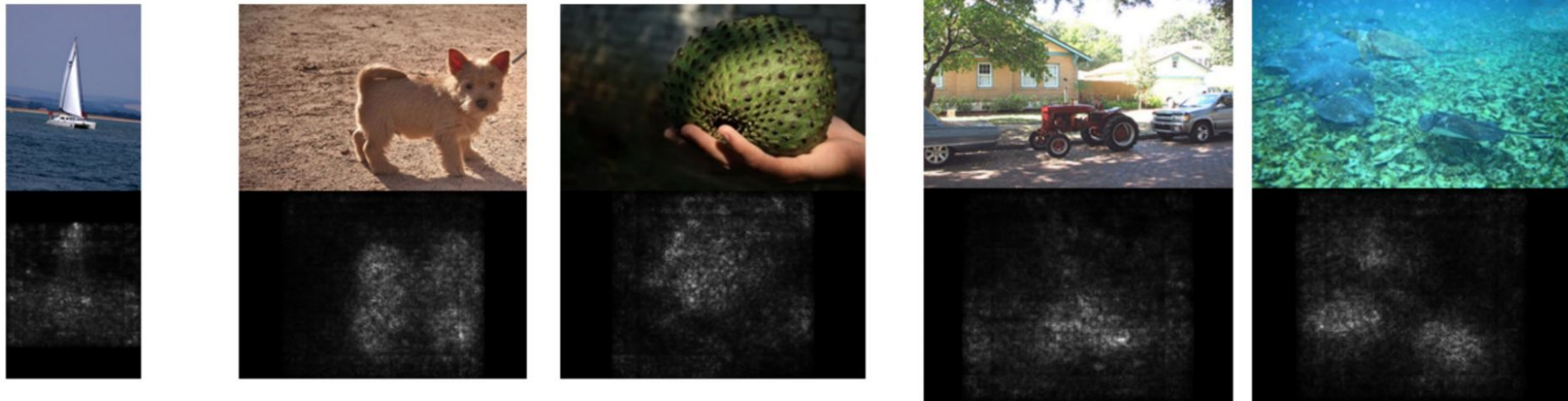


Dog

Compute gradient of (unnormalized) class score with respect to image pixels, take absolute value and max over RGB channels



Saliency Maps



Adversarial examples

African elephant



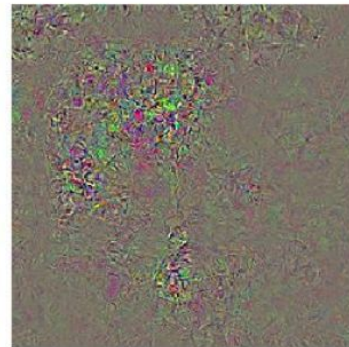
koala



Difference



10x Difference



schooner



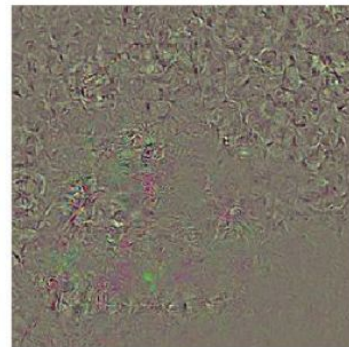
iPod



Difference



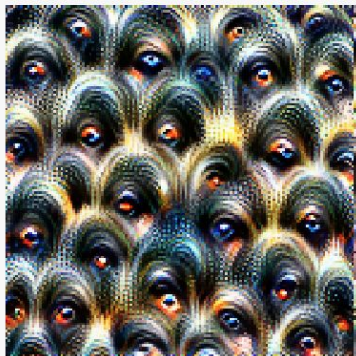
10x Difference



Feature Visualization

Demo

- Feature visualization



Positive optimized



Maximum activation
examples



Slightly positive
activation examples

Transfer Learning

- Training a ResNet-50 from scratch requires ~1.2 Million images (ImageNet) to converge well.
- In the real world (Medical, Manufacturing, Agriculture), you might only have 500 images.
- **The Trap:** If you train a deep network on small data, you get Overfitting immediately. The model memorizes the training set and fails on the test set.
- **The Solution:** Don't start from zero. Stand on the shoulders of giants.

Transfer Learning

- Idea: A network trained to recognize dogs and cats (ImageNet) has already learned how to detect edges, textures, circles, and fur.
- These "low-level features" are universal.
- We can reuse the "Knowledge" (Weights) from a massive task to solve a smaller, specific task.

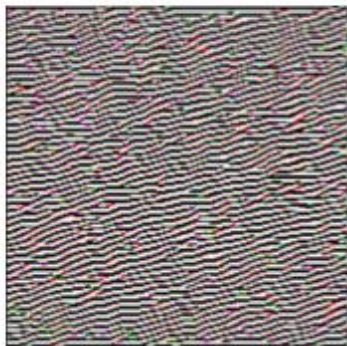
Transfer Learning

What actually gets transferred?

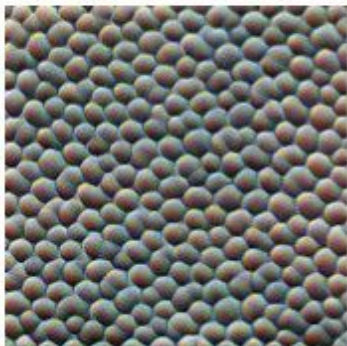
- **Layer 1-2:** Gabor Filters (Edges, color gradients). *These are identical for almost all vision tasks.*
- **Layer 10-20:** Texture detectors (Fur, woven fabric, eyes). *Highly reusable.*
- **Layer 40+:** Object parts (Dog face, Wheel). *Less reusable if your task is X-Rays.*

Transfer Learning

Edges



Textures



Patterns



Parts



Objects



Transfer Learning

- We conceptually split a CNN into two parts:
 1. **The Backbone (The Eye):** The Convolutional layers (ResNet, VGG). Its job is to turn pixels into meaningful features.
 2. **The Head (The Brain):** The Fully Connected (Linear) layers. Its job is to make a decision based on those features.
- **In Transfer Learning:** We usually keep the "Eye" and replace the "Brain."

Transfer Learning

Strategy 1: Feature Extraction (The "Freeze" Method)

Method:

1. Load a pre-trained model (e.g., ResNet-18).
2. **Freeze** all convolutional weights (gradient descent will NOT update them).
3. Delete the old classification head (1000 classes).
4. Add a new head for your task (e.g., 2 classes: Hotdog / Not Hotdog).
5. Train *only* the new head.

Pros: Very fast, impossible to "break" the pre-trained features.

When to use: Small dataset, similar domain to ImageNet.

Transfer Learning

Strategy 2: Fine-Tuning (The "Thaw" Method)

Method:

1. Start with Feature Extraction (train the head first).
2. **Unfreeze** the last few convolutional blocks.
3. Train with a very low learning rate (e.g., $1e-5$).

Why? Allows the "Eye" to adapt slightly to your specific data (e.g., adapting from "fur texture" to "X-ray texture").

Risk: Can lead to overfitting if data is too small.

Transfer Learning

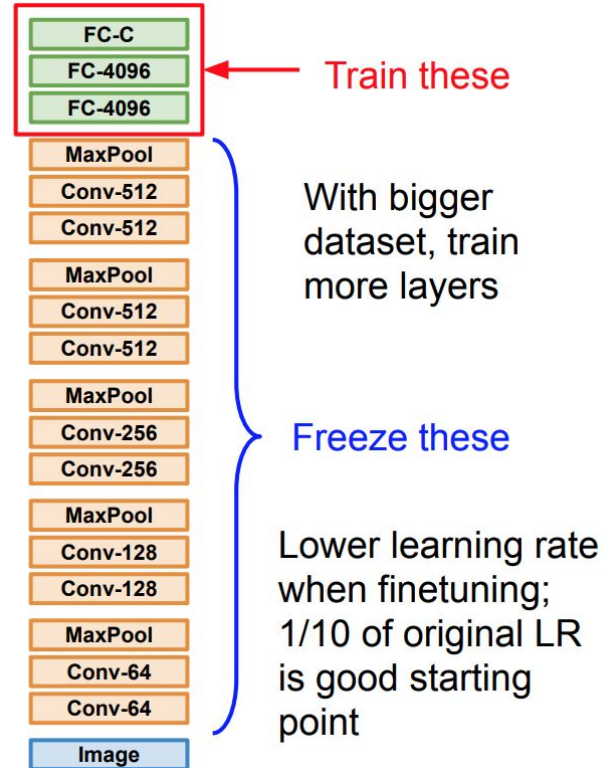
1. Train on Imagenet



2. Small Dataset (C classes)



3. Bigger dataset



Transfer Learning

What would YOU do?

Scenario A: You have 100 images of different car brands. You want to classify them.

Transfer Learning

What would YOU do?

Scenario A: You have 100 images of different car brands. You want to classify them.

Answer: Feature Extraction. (Data is too small to fine-tune; domain is very similar to ImageNet).

Transfer Learning

What would YOU do?

Scenario A: You have 100 images of different car brands. You want to classify them.

Answer: Feature Extraction. (Data is too small to fine-tune; domain is very similar to ImageNet).

Scenario B: You have 10,000 images of Satellite Imagery (Forest vs. Desert).

Transfer Learning

What would YOU do?

Scenario A: You have 100 images of different car brands. You want to classify them.

Answer: Feature Extraction. (Data is too small to fine-tune; domain is very similar to ImageNet).

Scenario B: You have 10,000 images of Satellite Imagery (Forest vs. Desert).

Answer: Fine-Tuning. (Data is decent size; domain (Satellite) is very different from ImageNet (Selfies/Dogs), so the "Eye" needs adjustment).

Scenario C: You have 10 million images of handwritten text.

Answer: Train from scratch. (Data is sufficient; domain is totally unique).

Receptive Fields

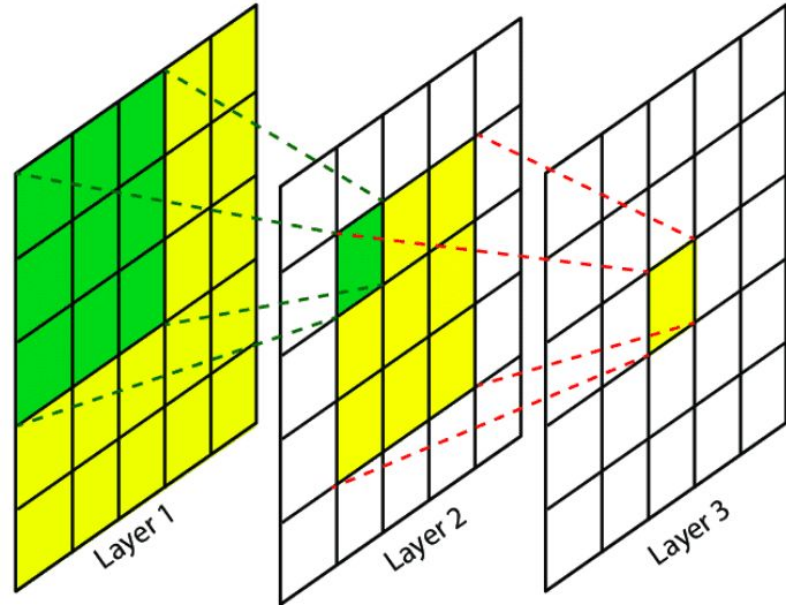
- A 3x3 kernel only looks at 9 pixels.
- How does the network recognize a large object (like a building) if it only looks at tiny patches?
- **Answer:** Stacking layers increases the "Receptive Field."

Receptive Fields

- **Layer 1:** 1 pixel sees 3×3 input pixels.
- **Layer 2:** 1 pixel sees 3×3 from Layer 1... which means it effectively sees 5×5 from the Input!
- **Layer 3:** Sees 7×7 .
- **Concept:** As we go deeper (pooling and striding), the Receptive Field grows exponentially.

Receptive Fields

By focusing on small regions of the input at a time and sharing parameters across these regions, CNNs can effectively capture local patterns and spatial relationships in the data. This allows CNNs to achieve translation invariance, meaning they can recognize objects regardless of their position within the input image.



Example

Can you tell what this is?



Example

How about now?



Example

How about now?



Receptive Fields

- Imagine detecting a "Skyscraper."
- If your RF is only 50x50 pixels, the network sees a patch of gray window.
- Is it a skyscraper? A gray car? A gray road?
- **Context is King:** To identify the object, you often need to see the *surroundings* (the sky above, the street below).
- *DeepLab* (Google) solved this using Dilated Convolutions to expand RF to the whole image.

Receptive Fields

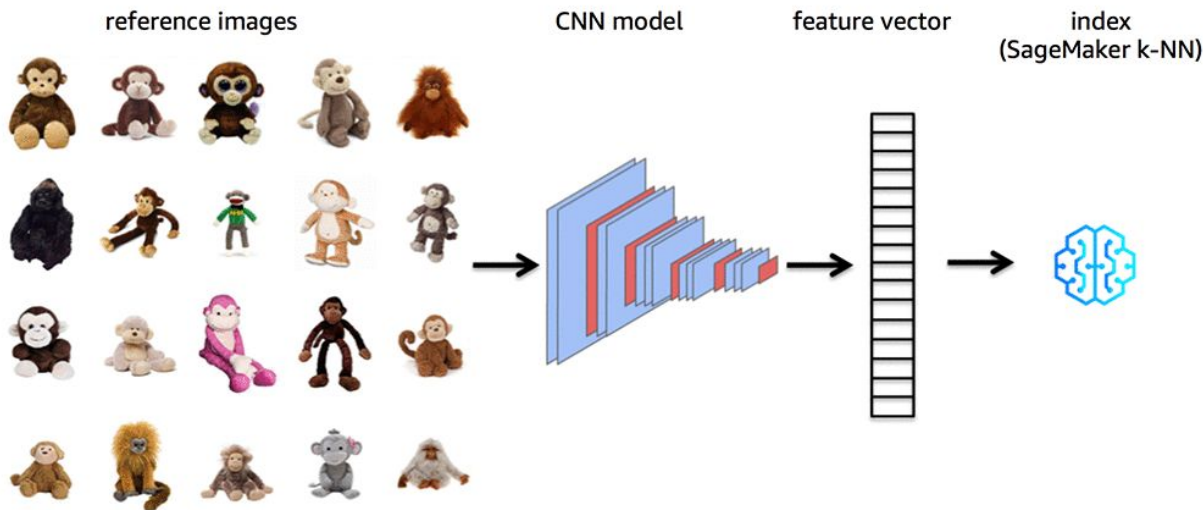
- If your Receptive Field is smaller than the object you are trying to detect, the network will fail.
- *Example:* Trying to classify a "Dog" but the network only sees "Fur" (it doesn't know if it's a cat, dog, or carpet).
- **Solution:** We need deep networks or tricks (Dilated Convolutions) to see the whole image.

Embeddings & Image Search

- What happens right before the final classification layer?
- The network has compressed the $224 \times 224 \times 3$ image (150k numbers) into a single vector of size 512 (the **Embedding**).
- This vector represents the "Essence" of the image.
- **Idea:** If we remove the classification head, we turn the CNN into a "Feature Extractor."

Embeddings & Image Search

- Images that look similar *visually* should be close to each other *mathematically* in this 512-dimensional space.
- All "Dogs" cluster in one corner; all "Cars" in another.
- **Application:** Similar Image Search (Reverse Image Search).



Embeddings & Image Search

How do we measure "Similarity"?

- We have two vectors: A (Query Image) and B (Database Image).
- **Euclidean Distance (L2):** $\sqrt{[(\sum A_i - B_i)^2]}$. Measures physical distance. Good, but sensitive to magnitude.
- **Cosine Similarity:** $\frac{A \cdot B}{||A|| \cdot ||B||}$ Measures the **angle** between vectors.
- *Rule of Thumb:* In Deep Learning, **Cosine Similarity** is usually better for embeddings.

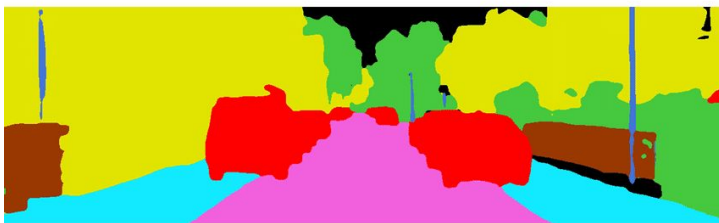
Embeddings & Image Search

How to Search 1 Billion Images?

- Comparing Query vs. 1 Database Image = Fast.
- Comparing Query vs. 1,000,000 Images = Slow (Linear Search).
- **Vector Databases (FAISS, Pinecone, Milvus):**
- They use indexing (like KD-Trees or HNSW) to find the nearest neighbor in milliseconds without checking every image.

Semantic Segmentation

- **Classification:** "This image contains a cat." (1 output)
- **Detection:** "The cat is in this box." (4 outputs: x, y, w, h)
- **Segmentation:** "Which pixels belong to the cat?" (Output size = Input size).
- Applications: Self-driving cars (Road vs. Sidewalk), Medical Imaging (Tumor vs. Tissue).



 Road	 Sidewalk	 Building	 Fence
 Pole	 Vegetation	 Vehicle	 Unlabel

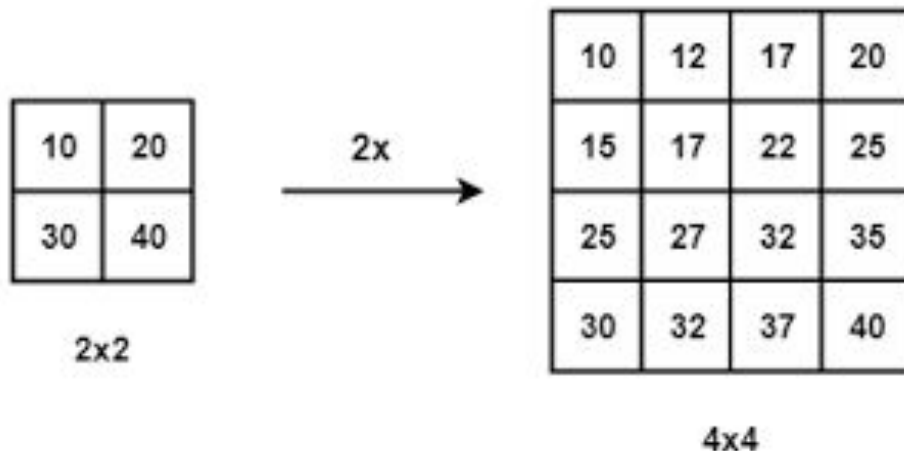
Semantic Segmentation

- In classification, we use Pooling to *shrink* the image (downsampling) to get high-level features.
- In segmentation, we need to output a full-resolution mask.
- **Challenge:** How do we go from a tiny 7x7 feature map back up to 224x224?

Upsampling methods

Method 1: Bilinear Interpolation

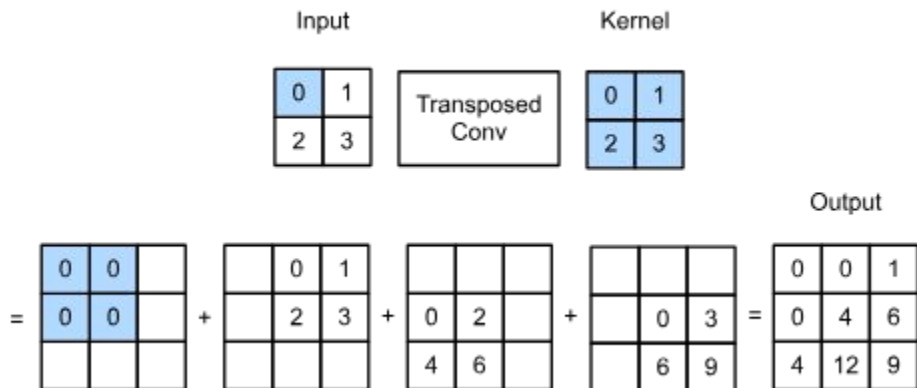
- **Mechanism:** Just math. Calculate the average of neighbors to fill in the gaps.
- **Pros:** Fast, no parameters to learn.
- **Cons:** Blurry output. The network doesn't "learn" how to reconstruct shape.



Upsampling methods

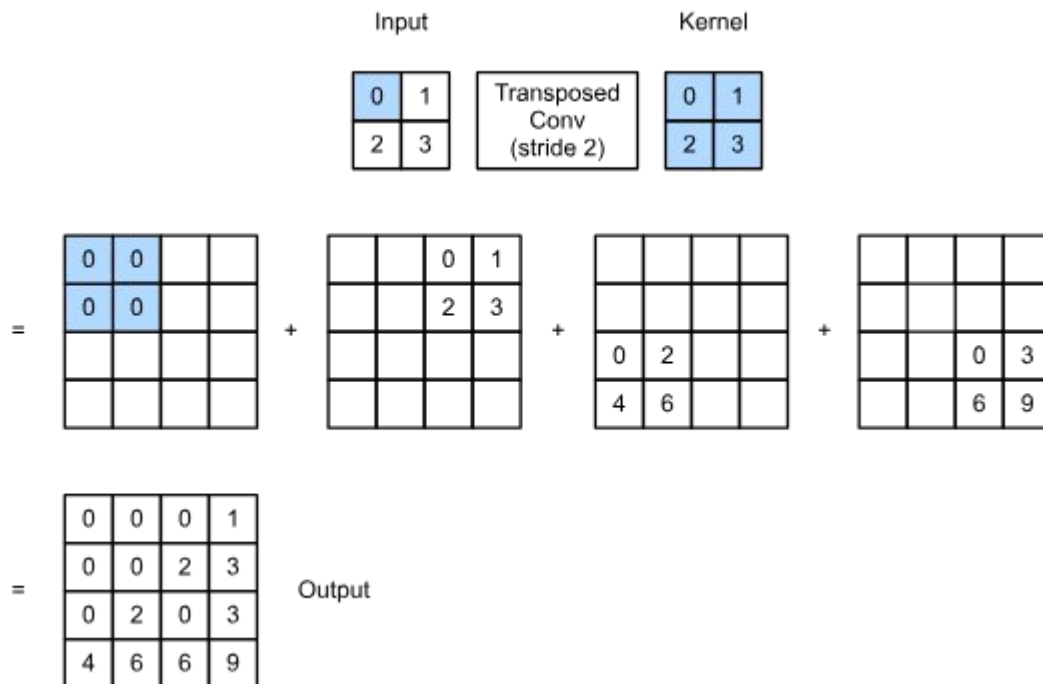
Method 2: Transposed Convolution (Deconvolution)

- **Mechanism:** A "Learnable Upsampling" layer.
- Instead of taking one value from a patch (Convolution), it takes one value and "pastes" a learnable kernel onto the output.
- Allows the network to learn *how* to upscale features intelligently.



Deconvolutions with stride

Method 2: Transposed Convolution (Deconvolution)



Problem with Deconvolution

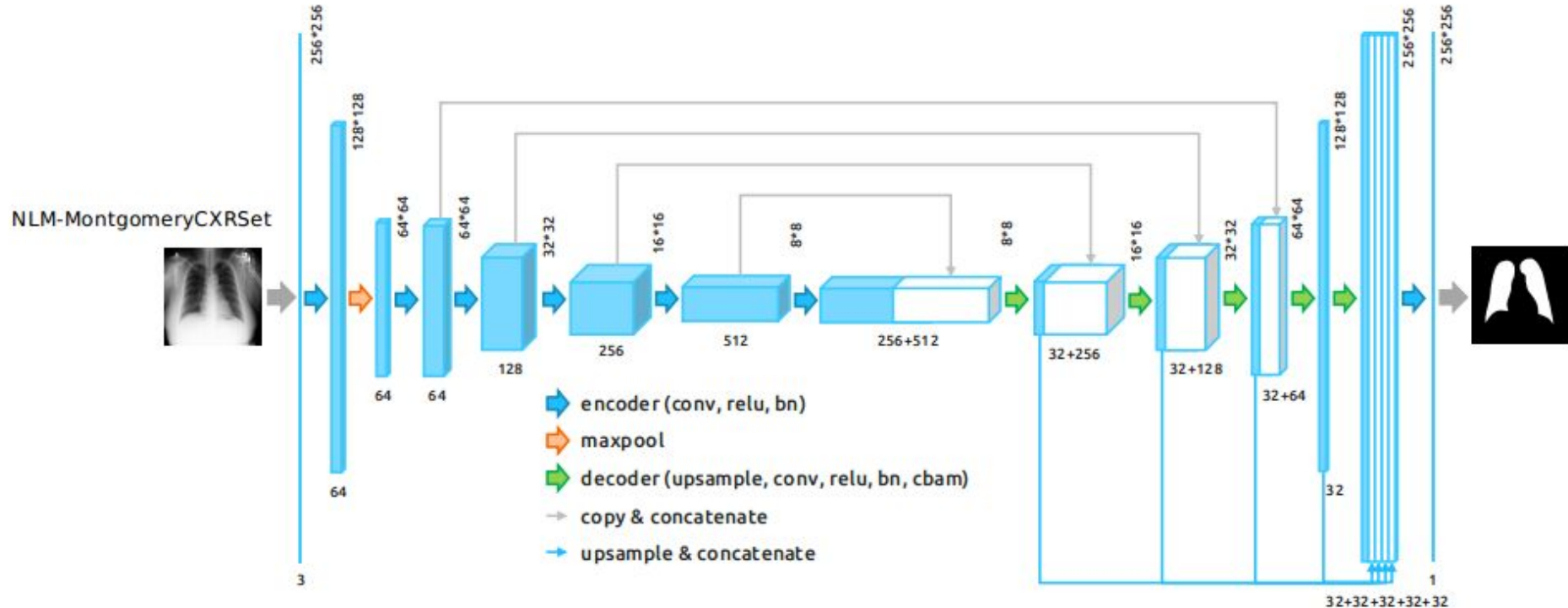
- Transposed Convolutions often overlap unevenly.
- This creates a visible "Checkerboard" pattern in the output mask.
- **Modern Fix:** Instead of Transposed Conv, many modern networks (like DeepLabV3) use:
 - `UpSample(scale_factor=2, mode='bilinear')` (Fixed math)
 - `Conv2d` (Learnable refinement)

Problem with Deconvolution



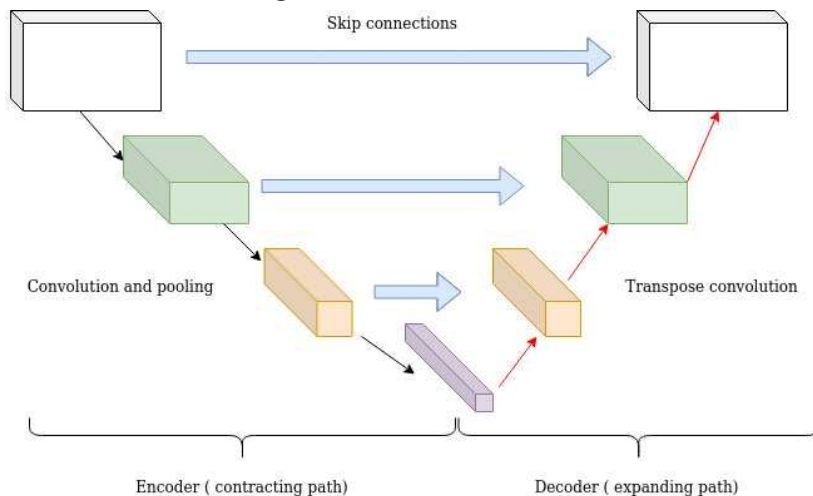
U-Net

Lung Segmentation:UNET Resnet34



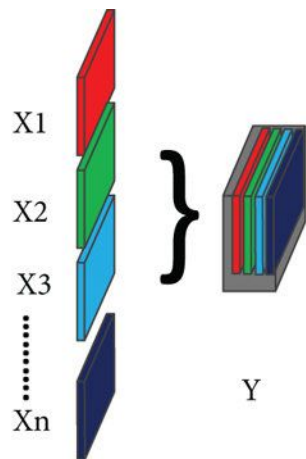
Skip Connections

- **Problem:** When we downsample to 7x7, we lose spatial information (exact edge locations).
- **Solution:** Copy the feature map from the Encoder level and **concatenate** it to the Decoder level.
- This gives the decoder the "High Res" details it needs to draw sharp boundaries.

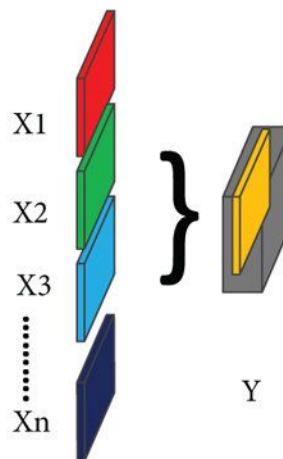


Add vs. Concatenate (ResNet vs UNet)

- **ResNet (Addition):** $F(x) + x$. "Superimposing" information. Keeps channel count the same. Used to flow gradients easier.
- **UNet (Concatenation):** $[F(x), x]$. "Appending" information. Increases channel count. Used to combine *Context* (Deep) with *Texture/Location* (Shallow).



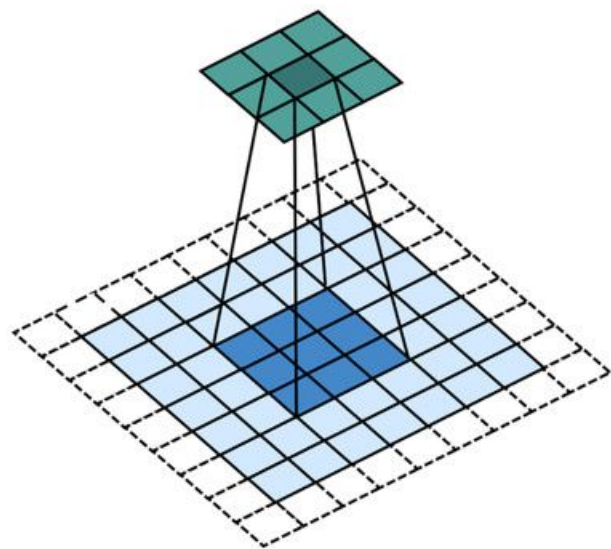
Concatenation operation



Addition operation

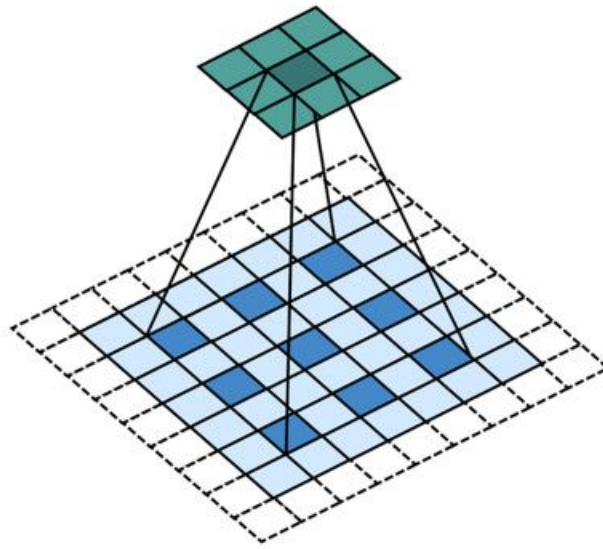
Dilated Convolutions

dilated=1



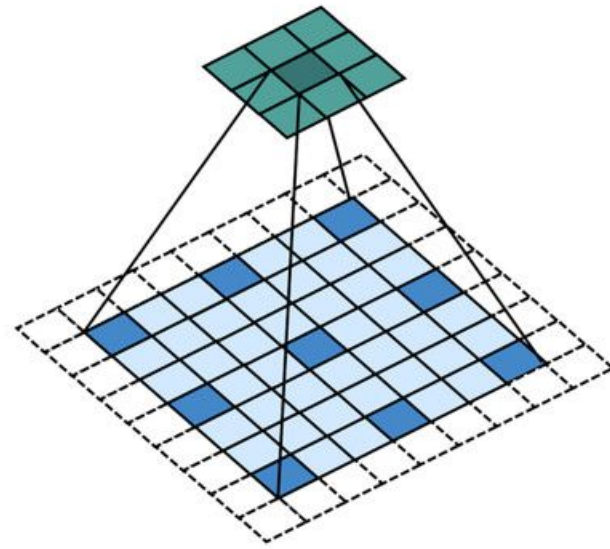
(a)

dilated=2



(b)

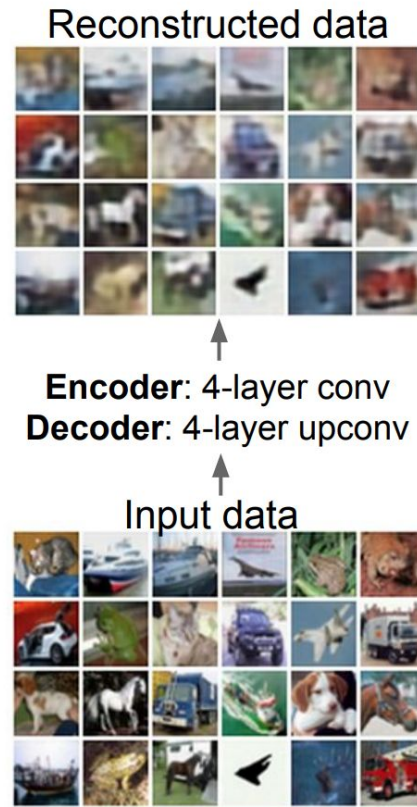
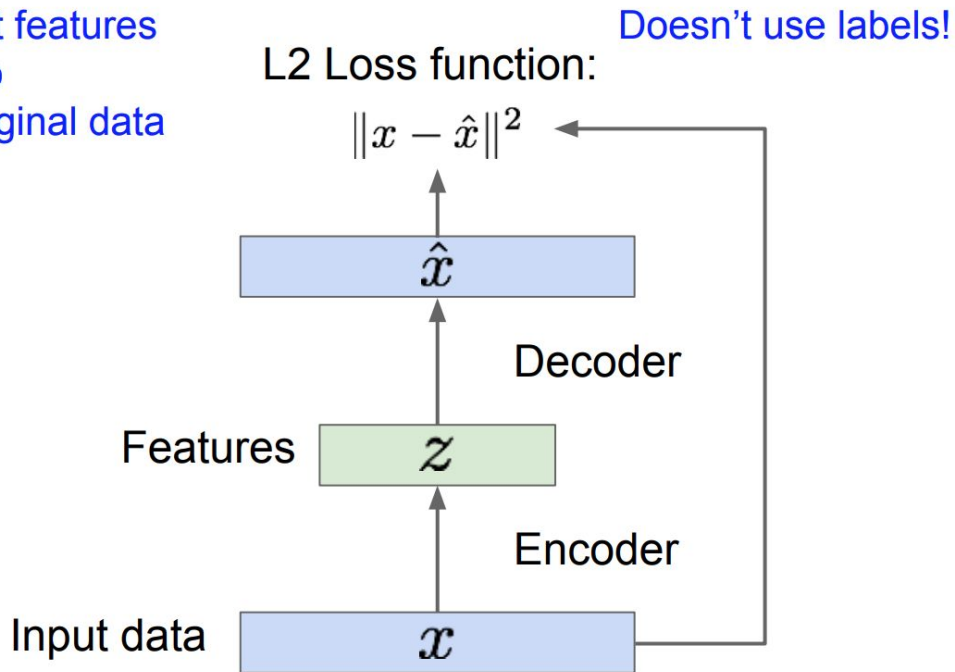
dilated=3



(c)

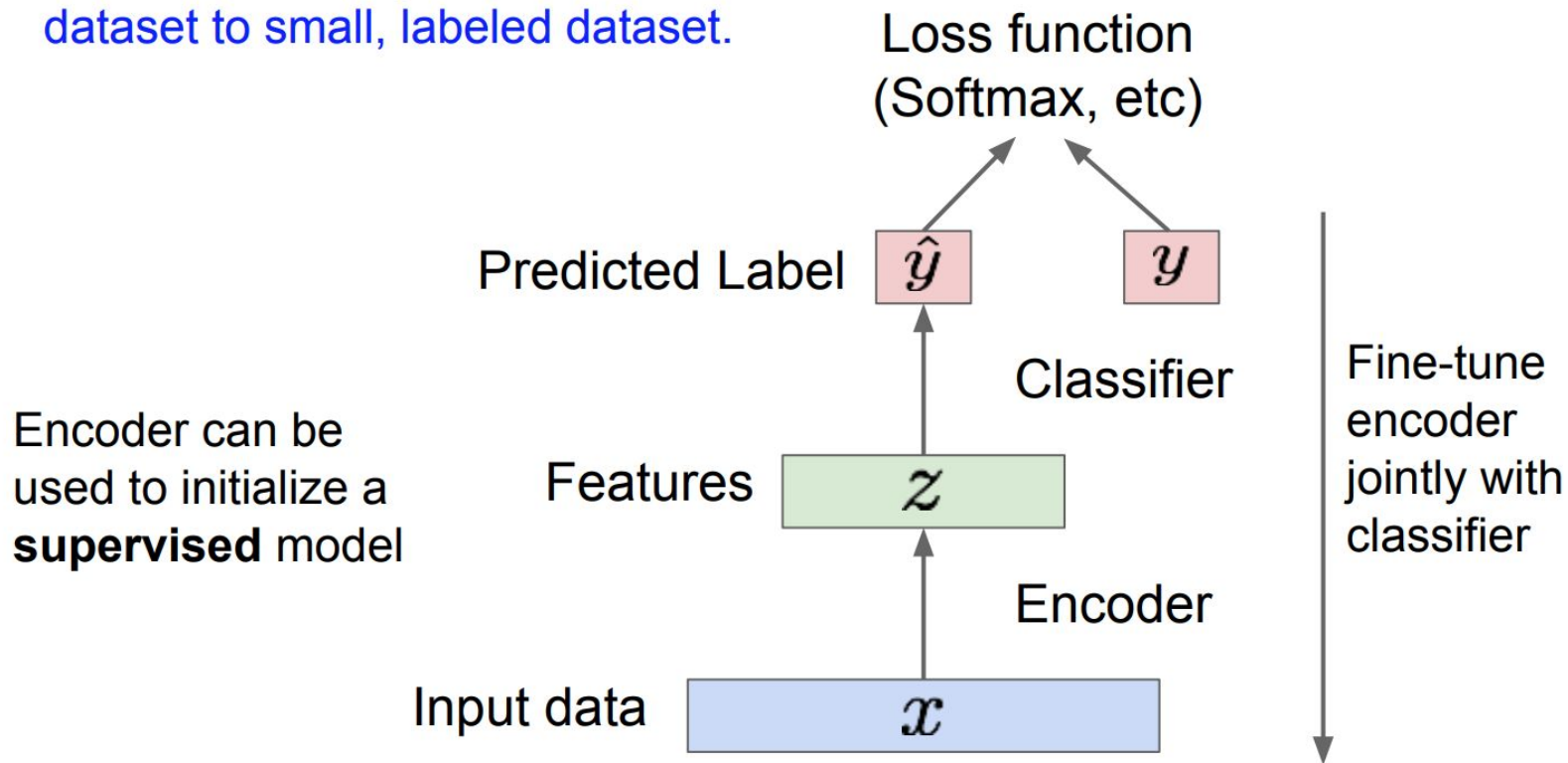
Autoencoders

Train such that features
can be used to
reconstruct original data



Autoencoders

Transfer from large, unlabeled dataset to small, labeled dataset.



Metrics

Multi-Class Precision & Recall

- In binary classification, Precision/Recall are simple.
- In **Multi-Class** (e.g., Dog vs. Cat vs. Background), we get a Precision/Recall *per class*. How do we summarize them into one number?
- **Micro-Average**: Calculate metrics globally (aggregating all True Positives).
 - *Effect*: Dominated by the majority class. (If "Background" is 90% of pixels, Micro-Score will be high even if you miss all objects).
- **Macro-Average**: Calculate metric for each class independently, then average them.
 - *Effect*: Treats all classes equally. (A mistake on a tiny "Tumor" class penalizes the score just as much as a mistake on the huge "Background" class).
- **Takeaway**: Use **Macro-Average** when minority classes matter!

Segmentation Metrics

Intersection over Union (IoU)

Accuracy counts *pixels*, but objects are *areas*.

Formula: Area of Overlap / Area of Union

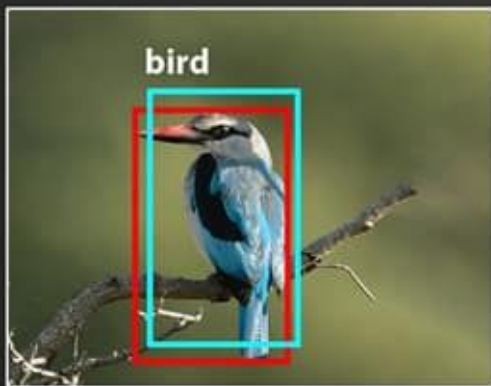
Why it's harder than Accuracy:

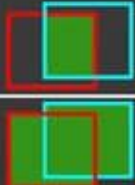
- To get IoU=1.0, you must predict the *exact* shape perfectly.
- A prediction that is slightly shifted might still have 90% accuracy (background matches) but only 0.5 IoU.

Mean IoU (mIoU): The standard metric for Segmentation leaderboards. We calculate IoU for *each class* (Road, Car, Pedestrian) and average them.

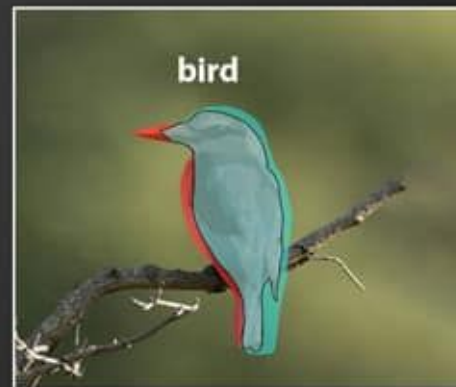
Intersection over Union

Intersection Over Union (IoU) in Object Detection and Segmentation



$$\text{IoU} = \frac{\text{Intersection}}{\text{Union}}$$


A diagram illustrating the components of the IoU formula. It shows two overlapping rectangles, one red and one cyan. The intersection of the two rectangles is shaded in green. The entire area covered by both rectangles (the union) is also indicated.



The Dice Coefficient (F1 for Pixels)

Concept: It is essentially the **F1 Score** applied to pixel sets.

Formula: $2 \times |A \cap B| / (|A| + |B|)$ (Harmonic Mean of Precision and Recall).

IoU vs. Dice:

- They are positively correlated (if one goes up, the other goes up).
- **Dice** is often used as a **Loss Function** (Soft Dice Loss) because it is differentiable and handles class imbalance better than Cross-Entropy for segmentation tasks (like medical imaging).

The Dice Coefficient (F1 for Pixels)

